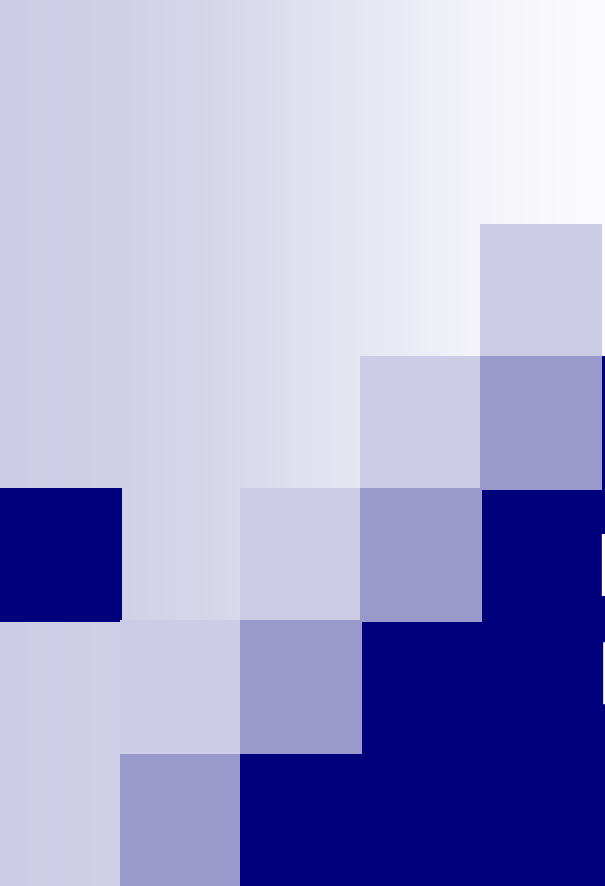




# Linguaggi di Programmazione

Docente: Cataldo Musto

## Capitolo 5 – Automi a stati finiti (Parte 1)

Si ringraziano il Prof. Giovanni Semeraro, il Prof. Marco de Gemmis e il Prof. Pasquale Lops per la concessione del materiale didattico di base

# Automi tra mito e realtà

- **Un automa** è una macchina in grado di operare in modo **autonomo**
  - Il termine automa deriva dal greco "automatos", "che agisce di propria volontà"
  - macchina semovente non elettronica (come un automa meccanico)
  - nell'uso colloquiale, comportarsi come un "automa" significa agire in modo meccanico, senza pensare

# I primi automi

- 1206: Al-Jazari, matematico, inventore e ingegnere meccanico arabo, realizza il primo automa **programmabile**
  - nave con quattro musicisti che galleggiava su un lago per intrattenere gli ospiti alle feste di corte
  - meccanismo basato su una batteria di percussioni **programmabile** con pistoncini che battevano su piccole leve che operavano la percussione
  - si potevano eseguire diverse partiture **posizionando opportunamente i pistoncini => programma**

# I primi automi

- 1495 circa: Leonardo da Vinci progettò un automa più complesso, detto *automa cavaliere*
  - cavaliere meccanico in armatura apparentemente in grado **di alzarsi in piedi, agitare le braccia e muovere testa e mascella** in modo anatomicamente corretto



## Il concetto di «stato»

- In sistemi di questo tipo, si può dire che in ogni istante **ci si trova in uno *stato*** preso da un insieme finito
  - × Uno stato ha lo scopo di *ricordare* una parte pertinente della storia del sistema e a determinare quali sono le possibili azioni che si possono effettuare in quel momento
  - × Avendo un numero finito di stati, generalmente non si può ricordare l'intera storia e quindi il sistema **va progettato attentamente affinché ricordi ciò che è importante e dimentichi ciò che non lo è**

# Un primo semplice esempio di automa

- Un interruttore
- *Quali sono i suoi possibili stati?*

# Un primo semplice esempio di automa

- Un interruttore
- *Quali sono i suoi possibili stati?*
  - Acceso e Spento
- *Quali operazioni si possono fare su ciascuno di questi stati?*

# Un primo semplice esempio di automa

- Un interruttore
- *Quali sono i suoi possibili stati?*
  - Acceso e Spento
- *Quali operazioni si possono fare su ciascuno di questi stati?*
  - Premere l'interruttore
- *Quale cambiamento di stato comporta l'aver effettuato l'operazione?*



# Un primo semplice esempio di automa

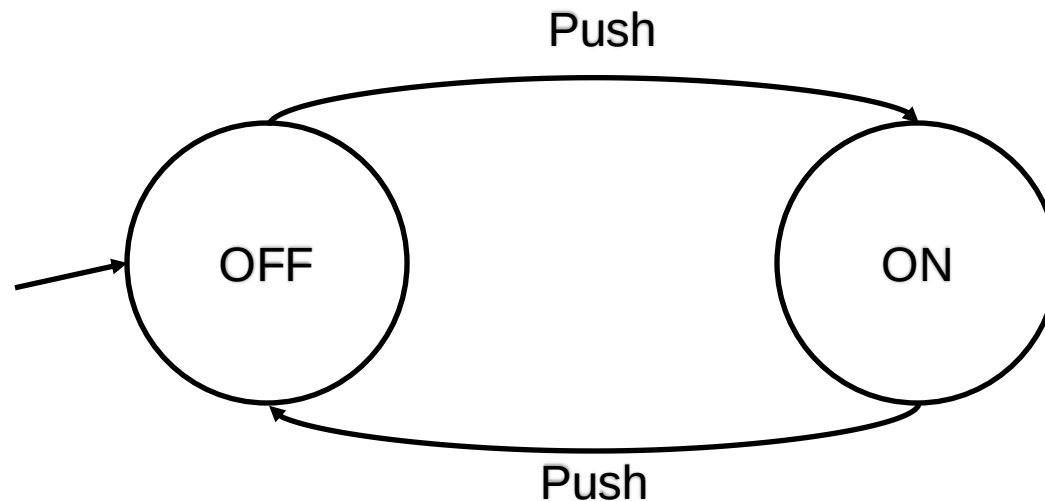
- Un interruttore
- *Quali sono i suoi possibili stati?*
  - Acceso e Spento
- *Quali operazioni si possono fare su ciascuno di questi stati?*
  - Premere l'interruttore
- *Quale cambiamento di stato comporta l'aver effettuato l'operazione?*
  - Se è acceso, si spegne. Se è spento, si accende (cambia lo stato)

# Un primo semplice esempio di automa

- **Stati** rappresentati per mezzo di cerchi
- **Archi** tra gli stati etichettati dagli input, che rappresentano le operazioni che il sistema può fare (push, che rappresenta la pressione del pulsante) e le conseguenze di tali operazioni

# Un primo semplice esempio di automa

- **Stati** rappresentati per mezzo di cerchi
- **Archi** tra gli stati etichettati dagli input, che rappresentano le operazioni che il sistema può fare (push, che rappresenta la pressione del pulsante) e le conseguenze di tali operazioni



# Un primo semplice esempio di automa

- **Uno degli stati è detto *iniziale***, cioè lo stato in cui il sistema si trova inizialmente
- Spesso è necessario indicare uno o più stati come ***finali*** o ***accettanti***. Se uno di questi viene raggiunto dopo una sequenza di input, tale sequenza si considera ***valida***. Per convenzione gli stati accettanti vengono rappresentati da un doppio cerchio

## Esempio automa per un parcheggio

- Fornire un automa a stati finiti che descriva il comportamento di un dispositivo che controlli un **parcheggio con due posti macchina dotato di una sbarra al suo ingresso.**
- Ci sono delle cellule fotoelettriche all'ingresso del parcheggio che permettono di sapere se una macchina ha attraversato un passaggio.

# Esempio automa per un parcheggio

- Fornire un automa a stati finiti che descriva il comportamento di un dispositivo che controlli un **parcheggio con due posti macchina dotato di una sbarra al suo ingresso**.
- Ci sono delle cellule fotoelettriche all'ingresso del parcheggio che permettono di sapere se una macchina ha attraversato un passaggio.
- Quali sono gli **stati**?
  - Cosa si serve memorizzare?

## Esempio automa per un parcheggio

- Fornire un automa a stati finiti che descriva il comportamento di un dispositivo che controlli un **parcheggio con due posti macchina dotato di una sbarra al suo ingresso**.
- Ci sono delle cellule fotoelettriche all'ingresso del parcheggio che permettono di sapere se una macchina ha attraversato un passaggio.
- Quali sono gli **stati**?
  - Il numero di auto dentro il parcheggio (0, 1, 2)

# Esempio automa per un parcheggio

- Fornire un automa a stati finiti che descriva il comportamento di un dispositivo che controlli un **parcheggio con due posti macchina dotato di una sbarra al suo ingresso**.
- Ci sono delle cellule fotoelettriche all'ingresso del parcheggio che permettono di sapere se una macchina ha attraversato un passaggio.
- Quali sono gli **input**?



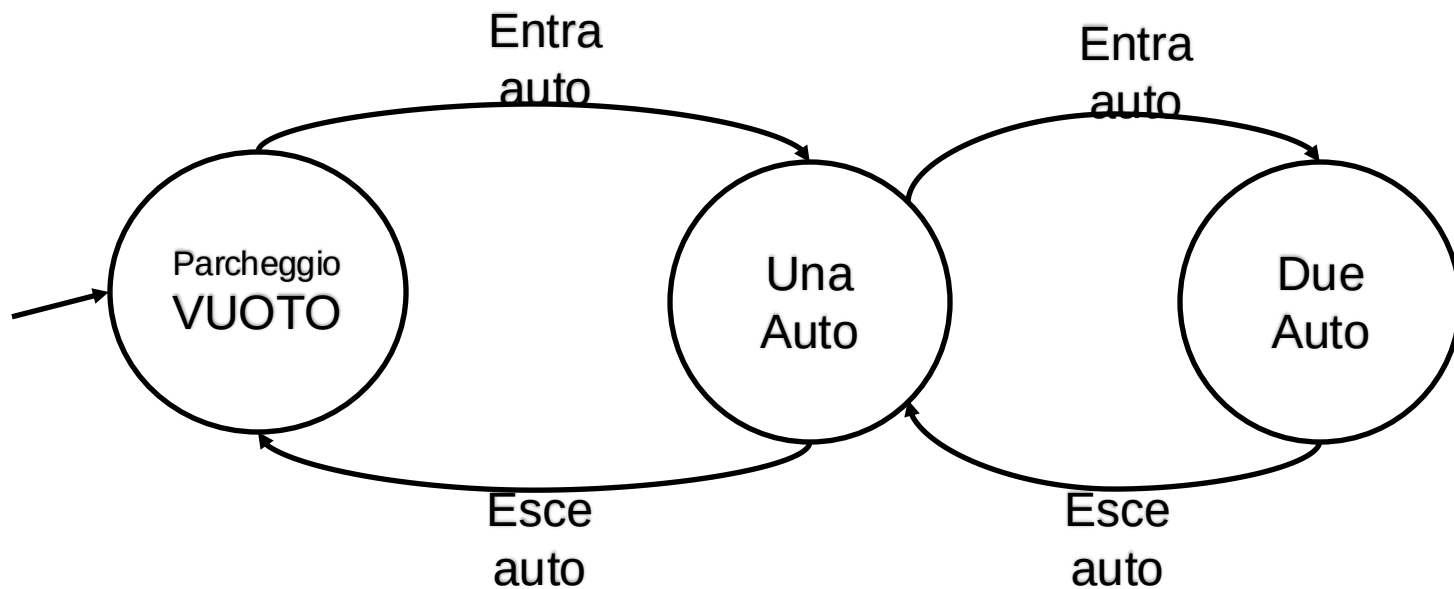
## Esempio automa per un parcheggio

- Fornire un automa a stati finiti che descriva il comportamento di un dispositivo che controlli un **parkeggio con due posti macchina dotato di una sbarra al suo ingresso.**
- Ci sono delle cellule fotoelettriche all'ingresso del parcheggio che permettono di sapere se una macchina ha attraversato un passaggio.
- Quali sono gli **input**?
  - Le auto, che possono entrare e uscire
- L'ingresso e/o uscita di ogni auto, genera un cambiamento nello stato



# Esempio automa per un parcheggio

# Esempio automa per un parcheggio



# Automi a Stati Finiti Deterministici

- Definizione – cosa è?
  - E' **un modello di calcolo** (relativamente semplice)
    - Una formalizzazione delle operazioni che può fare una macchina
  - A cosa servono, nell'ambito di questo insegnamento?

# Automi a Stati Finiti

- Linguaggi di Programmazione: a cosa ci servono?
  - Avevamo parlato di «**punto di vista riconoscitivo**»



# Automi a stati finiti

- Nell'insegnamento di linguaggi di programmazione studieremo un modello formale (=modello matematico) di automa a stati finiti
- **Scopriremo come progettare (=programmare) un automa per riconoscere un linguaggio L**
  - ✧ Data una stringa, l'automa risponde
    - SI = la stringa appartiene al linguaggio
    - NO oppure STOP se la parola non appartiene al linguaggio

# Automi a stati finiti

- Nell'insegnamento di linguaggi di programmazione studieremo un modello formale (=modello matematico) di automa a stati finiti
- **Scopriremo come progettare (=programmare) un automa per riconoscere un linguaggio L**
  - × Data una stringa, l'automa risponde
    - SI = la stringa appartiene al linguaggio
    - NO oppure STOP se la parola non appartiene al linguaggio
- **Sapremo rispondere al quesito**
  - × “Per quali linguaggi si può costruire un automa riconoscitore?”

# Automi a stati finiti deterministici

- Da cosa è composto un automa?
  - Un insieme di stati
  - Un insieme di simboli di input
  - Una funzione che definisce il suo comportamento



# Automi a stati finiti deterministici

- Da cosa è composto un automa?
  - **Un insieme di stati**
    - Situazioni da «ricordare»
    - Una 'traccia' della computazione effettuata fino a quel momento
  - Un insieme di entità di input
  - Una funzione che definisce il suo comportamento

# Automi a stati finiti deterministici

## ■ Da cosa è composto un automa?

### □ **Un insieme di stati**

- Situazioni da «ricordare»
- Una 'traccia' della computazione effettuata fino a quel momento
- Esempio: se l'interruttore è acceso o spento, oppure il numero di monete inserite!

### □ Un insieme di entità di input

### □ Una funzione che definisce il suo comportamento

# Automi a stati finiti deterministici

- Da cosa è composto un automa?
  - Un insieme di stati
  - **Un insieme di entità di input**
    - Ad esempio la pressione di un pulsante
    - Nel caso dei linguaggi, sono dei simboli (es. lettere)
  - Una funzione che definisce il suo comportamento

# Automi a stati finiti deterministici

- Da cosa è composto un automa?
  - Un insieme di stati
  - Un insieme di entità di input
  - **Una funzione che definisce il suo comportamento**
    - Definisce se lo 'stato' dell'automata cambia, in relazione a un particolare input ricevuto
    - **Nei nostri esempi, la funzione è deterministica**
- Formalizziamo!

# Automi a stati finiti deterministici

- Definizione di automa a stati finiti o accettore a stati finiti o FSA

Sia  $X$  un alfabeto. Un *automa a stati finiti* (FSA) è una quadrupla:

$$M = (Q, \delta, q_0, F)$$

# Automi a stati finiti deterministici

## ■ Definizione di automa a stati finiti o accettore a stati finiti o FSA

Sia  $X$  un alfabeto. Un *automa a stati finiti* (FSA) è una quadrupla:

$$M = (Q, \delta, q_0, F)$$

ove:

- $X$  è detto *alfabeto di ingresso*;
- $Q$  è un insieme finito e non vuoto di *stati*;
- $\delta$  è una funzione da in  $Q$ , detta *funzione di transizione*:

$$\delta : Q \times X \rightarrow Q$$

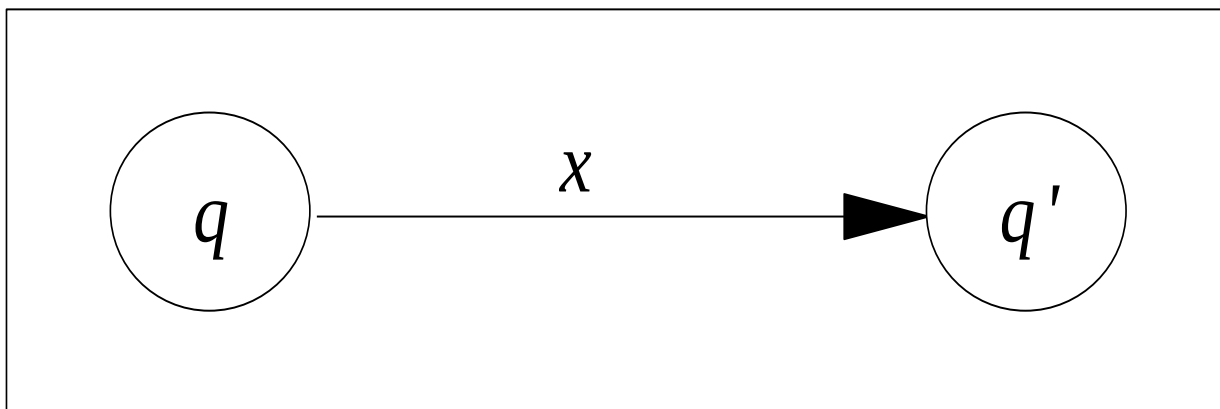
- $q_0$  è lo *stato iniziale*;
- $F \subseteq Q$  è l'insieme degli *stati di accettazione* o *finali*. 30/137

# Rappresentazione di FSA

- Un FSA può essere rappresentato mediante:
  - *Grafo degli Stati* o *Diagramma di Transizione* o *Diagramma di Stato*
    - ogni stato  $q \in Q$  è rappresentato da un cerchio (nodo) con etichetta  $q$ ;
    - lo stato iniziale (nodo  $q_0$ ) ha un arco orientato entrante libero (ossia, che non proviene da nessun altro nodo);
    - per ogni stato  $q \in Q$  e per ogni simbolo  $x$  dell'alfabeto di ingresso,  $x \in X$ , se  $\delta(q, x) = q'$  esiste un arco orientato etichettato con  $x$  uscente dal nodo  $q$  ed entrante nel nodo  $q'$ .

# Rappresentazione di FSA

- Un FSA può essere rappresentato mediante:
  - *Grafo degli Stati* o *Diagramma di Transizione* o *Diagramma di Stato*
    - ogni stato  $q \in Q$  è rappresentato da un cerchio (nodo) con etichetta  $q$ ;
    - lo stato iniziale (nodo  $q_0$ ) ha un arco orientato entrante libero (ossia, che non proviene da nessun altro nodo);
    - per ogni stato  $q \in Q$  e per ogni simbolo  $x$  dell'alfabeto di ingresso,  $x \in X$ , se  $\delta(q, x) = q'$  esiste un arco orientato etichettato con  $x$  uscente dal nodo  $q$  ed entrante nel nodo  $q'$ .





# Rappresentazione di FSA

- Un FSA può essere rappresentato mediante:

- *Tavola di Transizione*

È una tabella in cui sono riportati gli stati sulle righe e i simboli dell'alfabeto di ingresso sulle colonne.

Per ogni coppia (stato-ingresso) si legge nella tavola lo stato successivo:

| $\delta$ | $x_1$   | $x_2$   | ... | .... | $x_n$   |
|----------|---------|---------|-----|------|---------|
| $q_0$    | $q_0^1$ | $q_0^2$ | ... | ...  | $q_0^n$ |
| $q_1$    | $q_1^1$ | $q_1^2$ | ... | ...  | $q_1^n$ |
| ...      | ...     | ...     | ... | ...  | ...     |
| ...      | ...     | ...     | ... | ...  | ...     |
| $q_m$    | $q_m^1$ | $q_m^2$ | ... | ...  | $q_m^n$ |

# Rappresentazione di FSA

- Un FSA può essere rappresentato mediante:

- *Tavola di Transizione*

È una tabella in cui sono riportati gli stati sulle righe e i simboli dell'alfabeto di ingresso sulle colonne.

Per ogni coppia (stato-ingresso) si legge nella tavola lo stato successivo:

| $\delta$ | $x_1$   | $x_2$   | ... | .... | $x_n$   | Alfabeto |
|----------|---------|---------|-----|------|---------|----------|
| $q_0$    | $q_0^1$ | $q_0^2$ | ... | ...  | $q_0^n$ |          |
| $q_1$    | $q_1^1$ | $q_1^2$ | ... | ...  | $q_1^n$ |          |
| ...      | ...     | ...     | ... | ...  | ...     | Stati    |
| ...      | ...     | ...     | ... | ...  | ...     |          |
| $q_m$    | $q_m^1$ | $q_m^2$ | ... | ...  | $q_m^n$ |          |

# Rappresentazione di FSA

- Un FSA può essere rappresentato mediante:

- *Tavola di Transizione*

È una tabella in cui sono riportati gli stati sulle righe e i simboli dell'alfabeto di ingresso sulle colonne.

Per ogni coppia (stato-ingresso) si legge nella tavola lo stato successivo:

| $\delta$ | $x_1$   | $x_2$   | ... | .... | $x_n$   |
|----------|---------|---------|-----|------|---------|
| $q_0$    | $q_0^1$ | $q_0^2$ | ... | ...  | $q_0^n$ |
| $q_1$    | $q_1^1$ | $q_1^2$ | ... | ...  | $q_1^n$ |
| ...      | ...     | ...     | ... | ...  | ...     |
| ...      | ...     | ...     | ... | ...  | ...     |
| $q_m$    | $q_m^1$ | $q_m^2$ | ... | ...  | $q_m^n$ |

→ Funzione di Transizione  
 $\delta : Q \times X \rightarrow Q$

# Rappresentazione di FSA

- Un FSA può essere rappresentato mediante:

- *Tavola di Transizione*

È una tabella in cui sono riportati gli stati sulle righe e i simboli dell'alfabeto di ingresso sulle colonne.

Per ogni coppia (stato-ingresso) si legge nella tavola lo stato successivo:

| $\delta$ | $x_1$   | $x_2$   | ... | .... | $x_n$   |
|----------|---------|---------|-----|------|---------|
| $q_0$    | $q_0^1$ | $q_0^2$ | ... | ...  | $q_0^n$ |
| $q_1$    | $q_1^1$ | $q_1^2$ | ... | ...  | $q_1^n$ |
| ...      | ...     | ...     | ... | ...  | ...     |
| ...      | ...     | ...     | ... | ...  | ...     |
| $q_m$    | $q_m^1$ | $q_m^2$ | ... | ...  | $q_m^n$ |

dove l'alfabeto di ingresso e l'insieme degli stati sono rispettivamente:

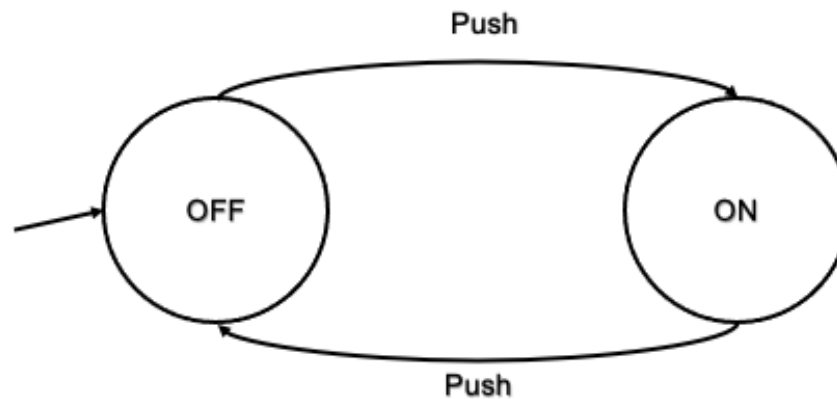
$$X = \{x_1, x_2, \dots, x_n\} \text{ e } Q = \{q_0, q_1, \dots, q_m\}$$

Quindi si ha che:

$$\delta(q_i, x_j) = q_i^j \text{ con } q_i, q_i^j \in Q, x_j \in X$$

# Rappresentazione di FSA - Esempio

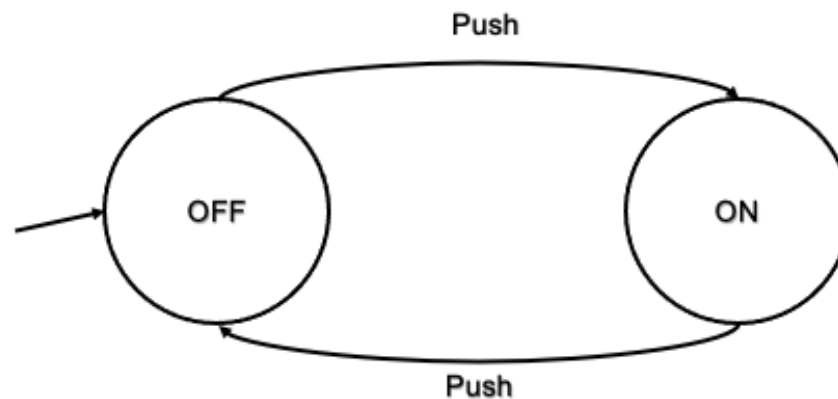
## ■ Tavola di Transizione



|     | Push |
|-----|------|
| OFF |      |
| ON  |      |

# Rappresentazione di FSA - Esempio

## ■ Tavola di Transizione



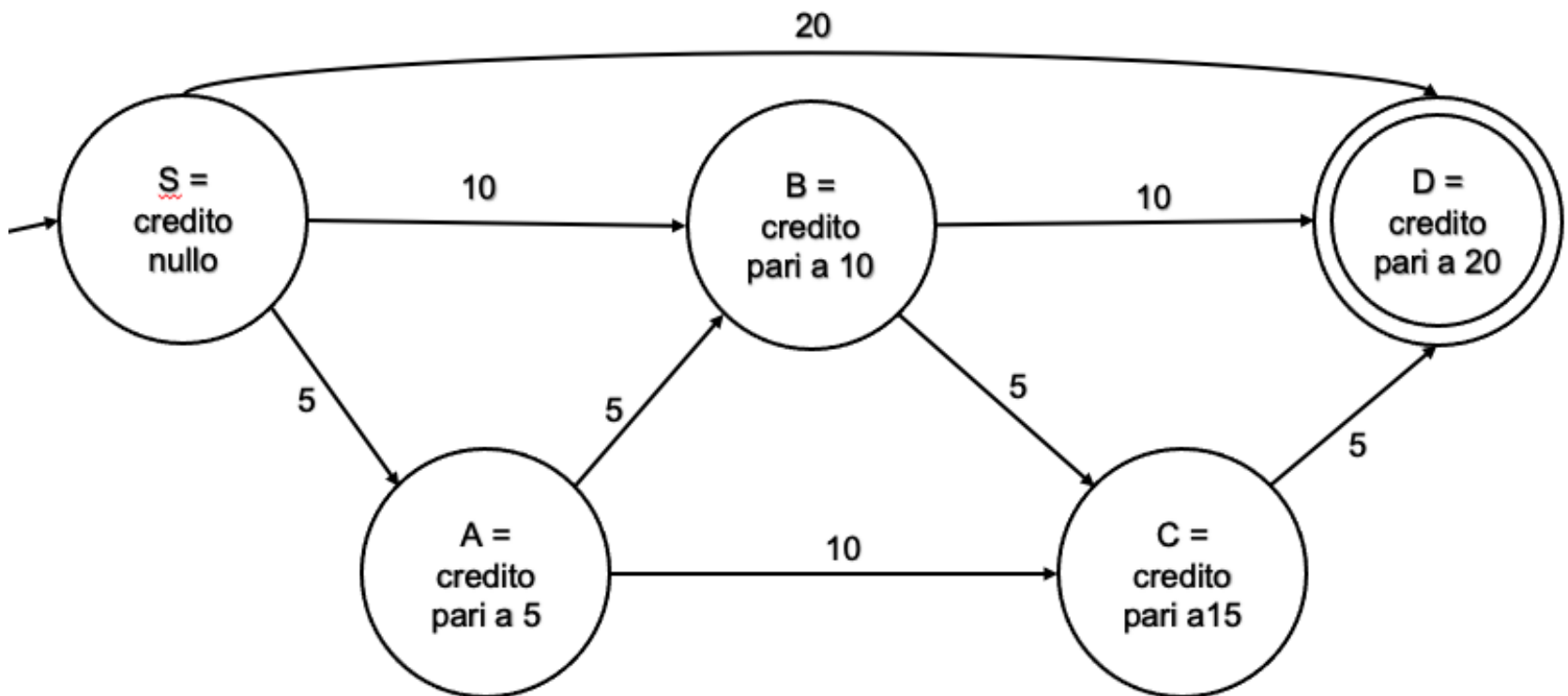
|     | Push |
|-----|------|
| OFF | ON   |
| ON  | OFF  |

# Automi a stati finiti deterministici

- Talora i valori della funzione di transizione  $\delta$  **non sono definiti per tutte le coppie** (stato-simbolo di ingresso)  $(q, x)$ . In tal caso, si dice che  $\delta$  è una **funzione parziale** o definita parzialmente.
- Questo significa che la lettura di  $x$  dà luogo in  $q$  ad un comportamento dell'automa che non si ritiene utile descrivere ai fini del riconoscimento (nel senso che produrrebbe stringhe non accettate).

# Automi a stati finiti deterministici

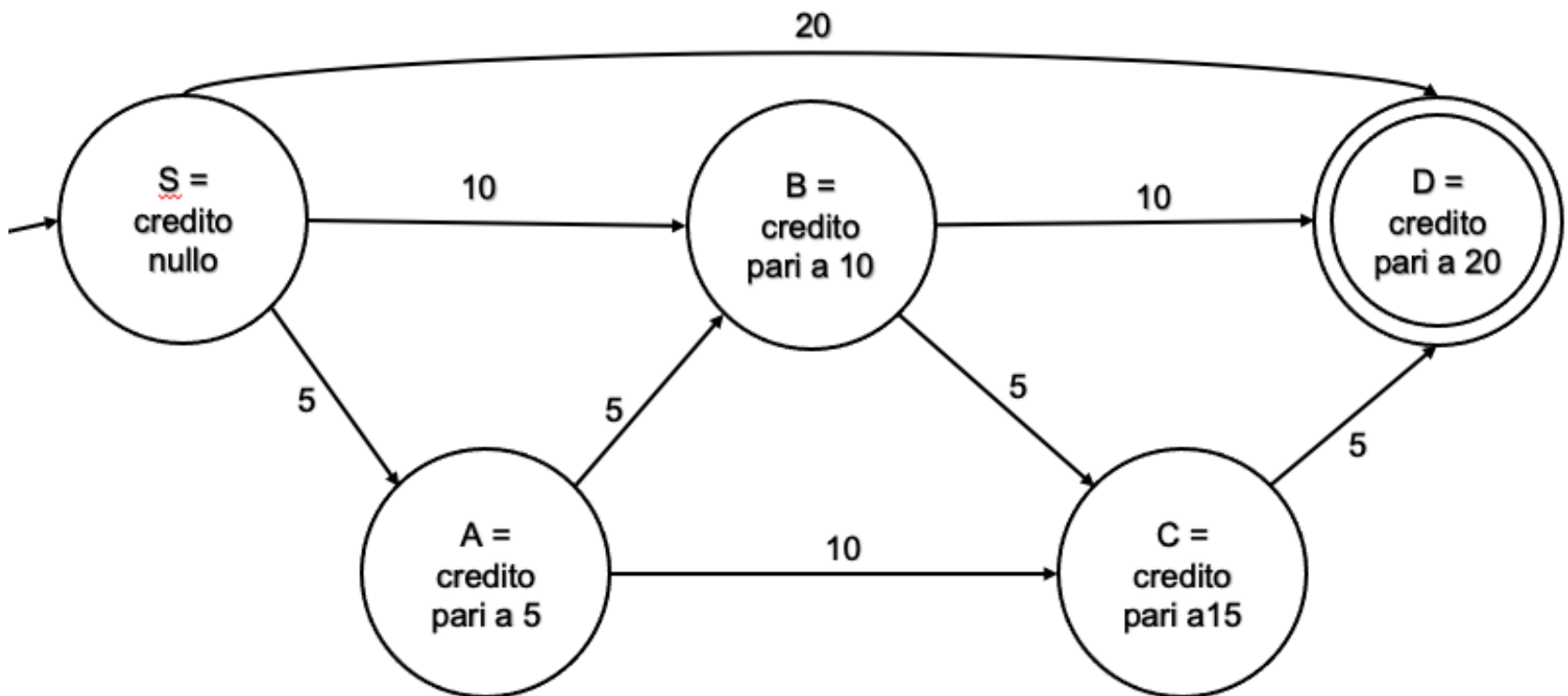
- L'immagine rappresenta un automa che registra il credito in un distributore automatico. I possibili input sono 5, 10 e 20 centesimi.





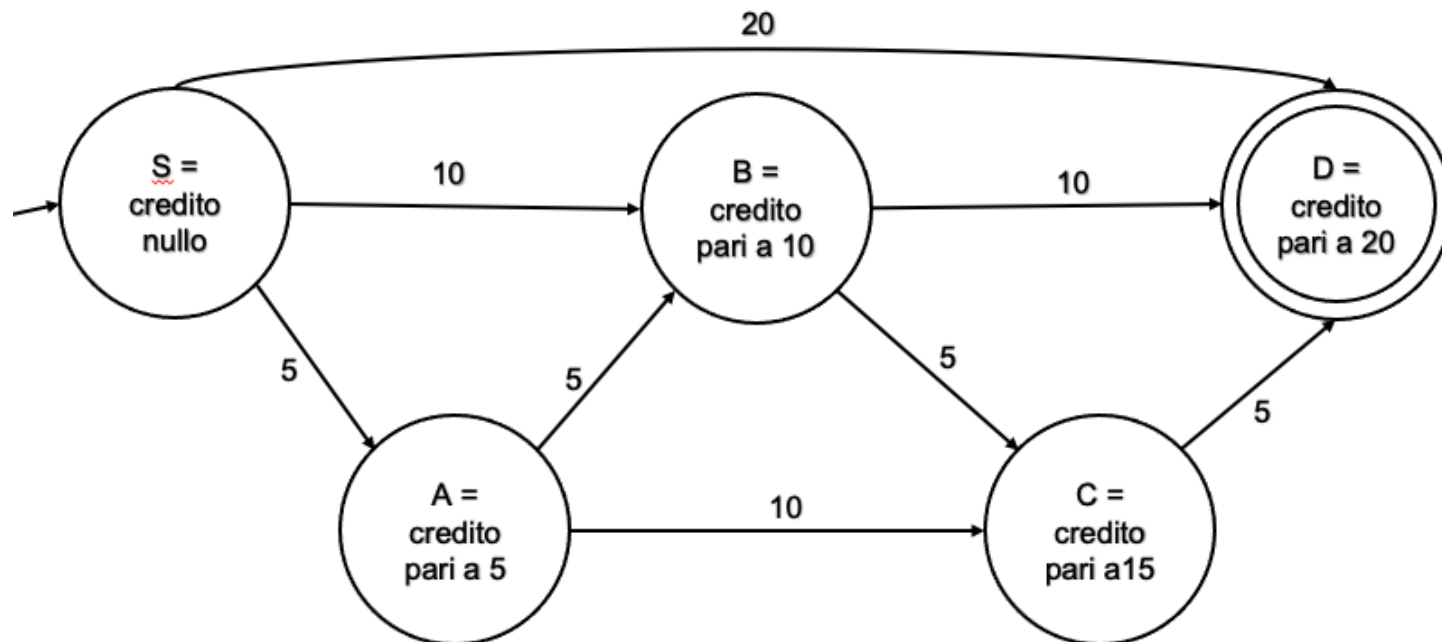
# Automi a stati finiti deterministici

- L'immagine rappresenta un automa che registra il credito in un distributore automatico. I possibili input sono 5, 10 e 20 centesimi. **Ci sono transizioni mancanti?**



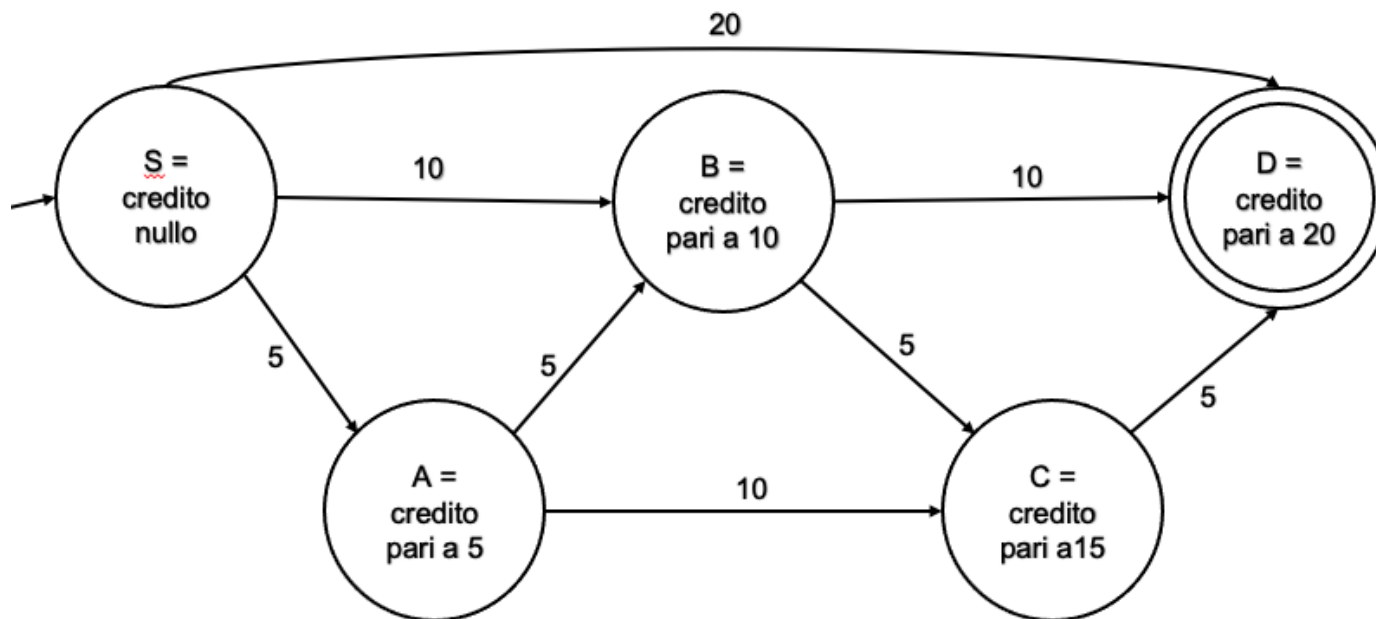
# Automi a stati finiti deterministici

- Ci sono transizioni mancanti?
  - Dallo stato A e B manca la transizione con input 20
  - Dallo stato C manca la transizione con input 10 e 20
  - **Perché?**



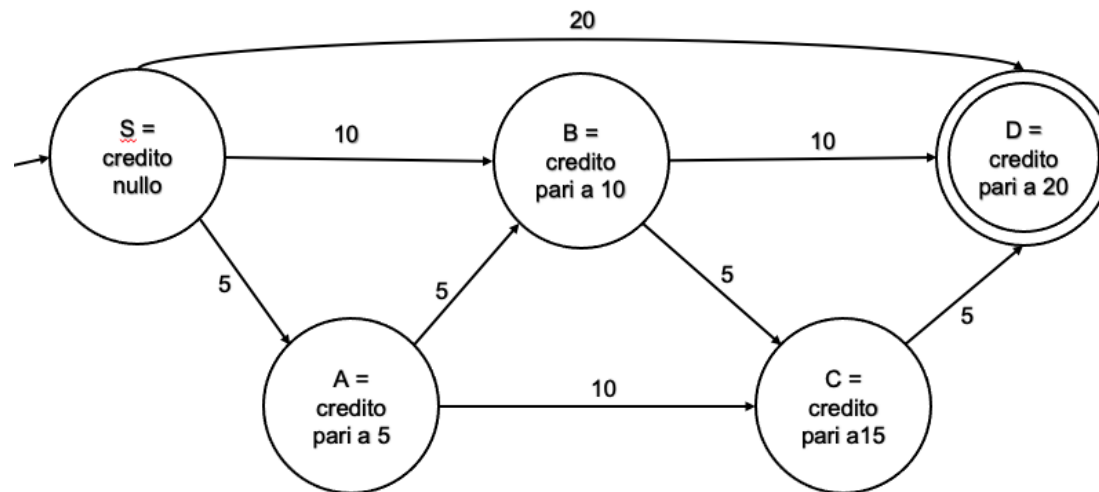
# Automi a stati finiti deterministici

- Ci sono transizioni mancanti?
  - Dallo stato A e B manca la transizione con input 20
  - Dallo stato C manca la transizione con input 10 e 20
  - **Perché? Perché ci porterebbe verso stati non validi**



# Automi a stati finiti deterministici

- La funzione può anche essere trasformata in **funzione totale** creando uno stato apposito (detto stato pozza) dove vanno a finire tutte le transizioni che non portano a una parola accettata dal linguaggio.



## Torniamo ai Linguaggi

- Immaginiamo di costruire un automa che riconosca il linguaggio  $a^n b^m$ , con  $n > 0$  e  $m \geq 0$ 
  - Quali sono le parole valide in questo linguaggio?

# Torniamo ai Linguaggi

- Immaginiamo di costruire un automa che riconosca il linguaggio  $a^n b^m$ , con  $n > 0$  e  $m \geq 0$ 
  - Sappiamo costruire una grammatica che generi questo linguaggio

## Torniamo ai Linguaggi

- Immaginiamo di costruire un automa che riconosca il linguaggio  $a^n b^m$ , con  $n > 0$  e  $m \geq 0$ 
  - Sappiamo costruire una grammatica che generi questo linguaggio

$$S \rightarrow a | aS | aB, B \rightarrow b | bB$$

# Torniamo ai Linguaggi

- Immaginiamo di costruire un automa che riconosca il linguaggio  $a^n b^m$ , con  $n > 0$  e  $m \geq 0$ 
  - Quali sono le parole valide in questo linguaggio?
    - Tutte le sequenze di 'a'
    - Parole che cominciano almeno con un a, seguite da un numero arbitrario di b

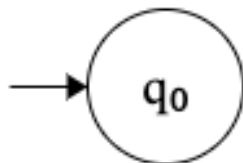


# Torniamo ai Linguaggi

- Immaginiamo di costruire un automa che riconosca il linguaggio  $a^n b^m$ , con  $n > 0$  e  $m \geq 0$ 
  - Quali sono le parole valide in questo linguaggio?
    - $L = \{a, aa, aaa, aaaa, ab, abb, aab, aabb, aaabb, abbbb \dots\}$

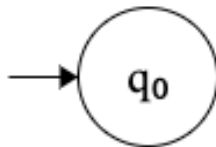
# Un esempio semplicissimo

- Immaginiamo di costruire un automa che riconosca il linguaggio  $a^n b^m$ , con  $n > 0$  e  $m \geq 0$ 
  - Quali sono le parole valide in questo linguaggio?
    - Tutte le sequenze di 'a'
    - Parole che cominciano almeno con un a, seguite da un numero arbitrario di b
  - Il processo di riconoscimento/creazione comincia creando uno stato (detto stato iniziale) tipicamente etichettato con  $q_0$



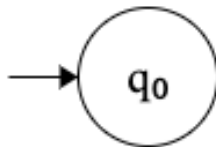
# Un esempio semplicissimo

- Immaginiamo di costruire un automa che riconosca il linguaggio  $a^n b^m$ , con  $n > 0$  e  $m \geq 0$ 
  - Per creare un automa, ragioniamo su quali simboli possono essere letti
    - Quali simboli possono essere letti?



# Un esempio semplicissimo

- Immaginiamo di costruire un automa che riconosca il linguaggio  $a^n b^m$ , con  $n > 0$  e  $m \geq 0$ 
  - Per creare un automa, ragioniamo su quali simboli possono essere letti
    - Quali simboli possono essere letti? 'a' e 'b'



# Un esempio semplicissimo

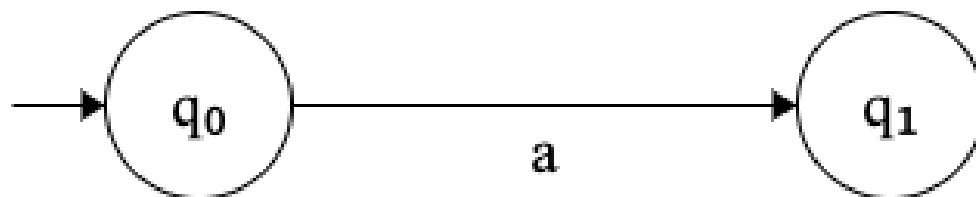
- Immaginiamo di costruire un automa che riconosca il linguaggio  $a^n b^m$ , con  $n > 0$  e  $m \geq 0$ 
  - Per creare un automa, ragioniamo su quali simboli possono essere letti
    - Quali simboli possono essere letti? **'a' e 'b'**
- Quando leggo un simbolo, posso fare due cose
  - **Mi sposto su un nuovo stato**, se ho bisogno di memorizzare quell'informazione
  - **Resto sullo stesso stato** (ciclo)

# Un esempio semplicissimo

- Immaginiamo di costruire un automa che riconosca il linguaggio  $a^n b^m$ , con  $n > 0$  e  $m \geq 0$ 
  - Per creare un automa, ragioniamo su quali simboli possono essere letti
    - Se leggo una 'a' ho esigenza di 'ricordarmi' di aver letto quel simbolo?

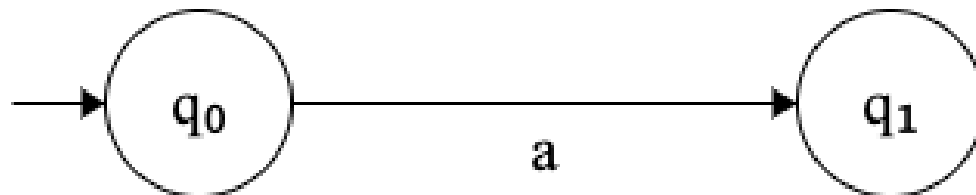
# Un esempio semplicissimo

- Immaginiamo di costruire un automa che riconosca il linguaggio  $a^n b^m$ , con  $n > 0$  e  $m \geq 0$ 
  - Per creare un automa, ragioniamo su quali simboli possono essere letti
    - Se leggo una 'a' ho esigenza di 'ricordarmi' di aver letto quel simbolo, quindi creo un nuovo stato



# Un esempio semplicissimo

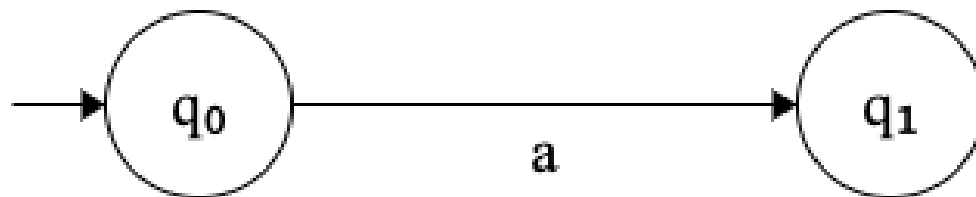
- Immaginiamo di costruire un automa che riconosca il linguaggio  $a^n b^m$ , con  $n > 0$  e  $m \geq 0$ 
  - Per creare un automa, ragioniamo su quali simboli possono essere letti
    - Se leggo una 'a' ho esigenza di 'ricordarmi' di aver letto quel simbolo, quindi creo un nuovo stato
    - Da  $q_0$  posso leggere 'b' ?





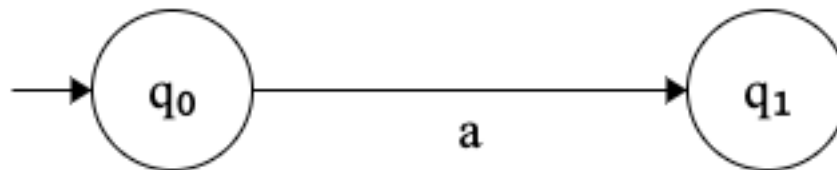
# Un esempio semplicissimo

- Immaginiamo di costruire un automa che riconosca il linguaggio  $a^n b^m$ , con  $n > 0$  e  $m \geq 0$ 
  - Per creare un automa, ragioniamo su quali simboli possono essere letti
    - Se leggo una 'a' ho esigenza di 'ricordarmi' di aver letto quel simbolo, quindi creo un nuovo stato
    - Da  $q_0$  posso leggere 'b' ? **No perché le parole non possono iniziare con b.**



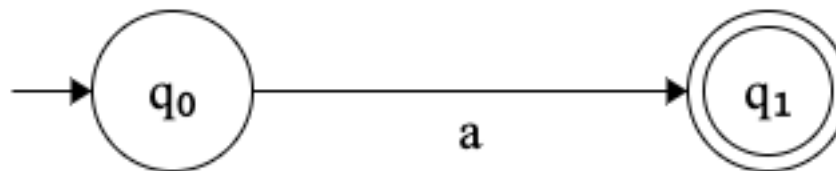
# Un esempio semplicissimo

- Immaginiamo di costruire un automa che riconosca il linguaggio  $a^n b^m$ , con  $n > 0$  e  $m \geq 0$ 
  - Per creare un automa, ragioniamo su quali simboli possono essere letti
    - Se leggo una 'a' ho esigenza di 'ricordarmi' di aver letto quel simbolo, quindi creo un nuovo stato
    - 'a' è una parola valida del linguaggio?



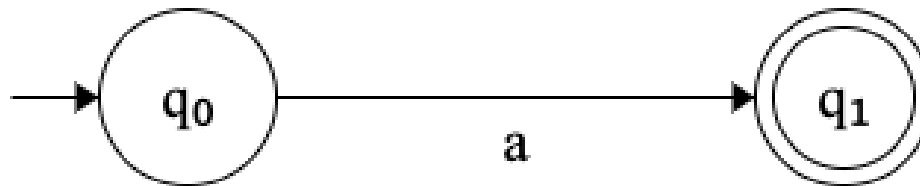
# Un esempio semplicissimo

- Immaginiamo di costruire un automa che riconosca il linguaggio  $a^n b^m$ , con  $n > 0$  e  $m \geq 0$ 
  - Per creare un automa, ragioniamo su quali simboli possono essere letti
    - Se leggo una 'a' ho esigenza di 'ricordarmi' di aver letto quel simbolo, quindi creo un nuovo stato
    - 'a' è una parola valida del linguaggio? Sì, dunque lo stato è anche uno stato «finale»



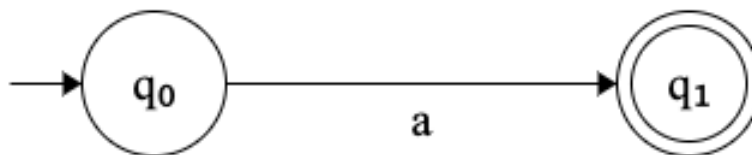
# Un esempio semplicissimo

- Immaginiamo di costruire un automa che riconosca il linguaggio  $a^n b^m$ , con  $n > 0$  e  $m \geq 0$ 
  - Per creare un automa, ragioniamo su quali simboli possono essere letti
    - Ripetiamo il processo anche sullo stato  $q_1$
    - Cosa succede se leggo 'a' ?
    - Cosa succede se leggo 'b' ?



# Un esempio semplicissimo

- Immaginiamo di costruire un automa che riconosca il linguaggio  $a^n b^m$ , con  $n > 0$  e  $m \geq 0$ 
  - Per creare un automa, ragioniamo su quali simboli possono essere letti
    - Ripetiamo il processo anche sullo stato  $q_1$
    - Cosa succede se leggo 'a' ?
      - Parole con ulteriori 'a' sono ancora parole valide.
      - Devo creare un nuovo stato per ogni 'a' che leggo?

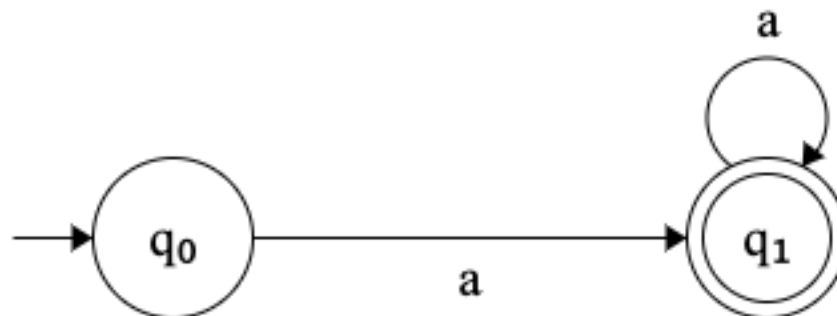


# Un esempio semplicissimo

- Immaginiamo di costruire un automa che riconosca il linguaggio  $a^n b^m$ , con  $n > 0$  e  $m \geq 0$ 
  - Per creare un automa, ragioniamo su quali simboli possono essere letti
    - Ripetiamo il processo anche sullo stato  $q_1$
    - Cosa succede se leggo 'a' ?
      - Parole con ulteriori 'a' sono ancora parole valide.
      - Devo creare un nuovo stato per ogni 'a' che leggo?
        - No, perché le 'a' possono essere potenzialmente infinite, mentre invece gli stati sono **finiti. E' sufficiente un ciclo, perché non mi serve ricordare quante sono.**
        - Basta che ce ne sia almeno una!

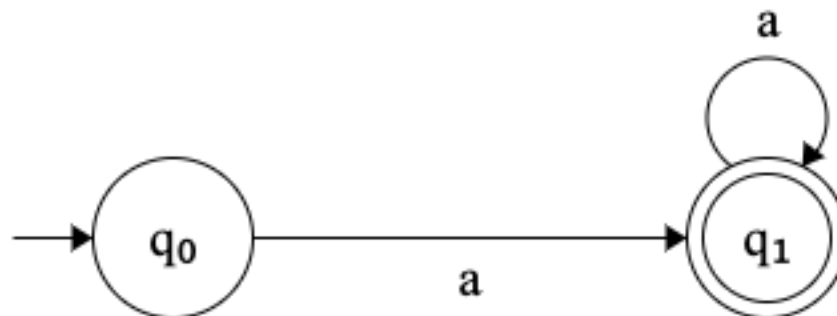
# Un esempio semplicissimo

- Immaginiamo di costruire un automa che riconosca il linguaggio  $a^n b^m$ , con  $n > 0$  e  $m \geq 0$ 
  - Per creare un automa, ragioniamo su quali simboli possono essere letti
    - Ripetiamo il processo anche sullo stato  $q_1$
    - Cosa succede se leggo 'a' ?
      - Ciclo sullo stato



# Un esempio semplicissimo

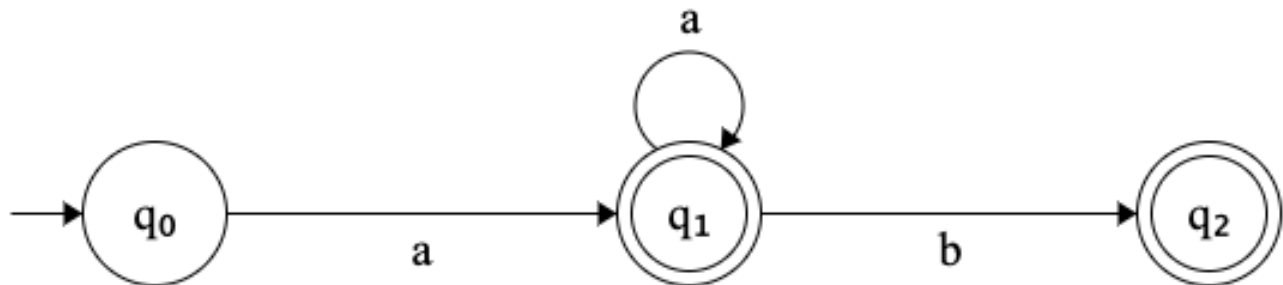
- Immaginiamo di costruire un automa che riconosca il linguaggio  $a^n b^m$ , con  $n > 0$  e  $m \geq 0$ 
  - Per creare un automa, ragioniamo su quali simboli possono essere letti
    - Ripetiamo il processo anche sullo stato  $q_1$
    - Cosa succede se leggo 'b' ?





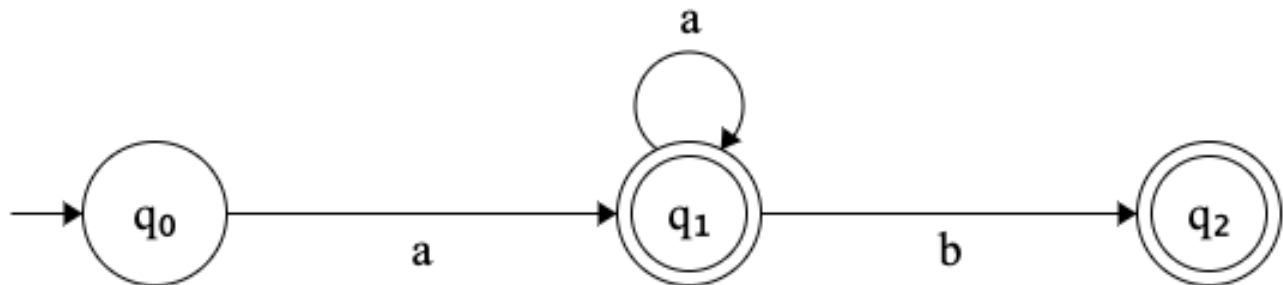
# Un esempio semplicissimo

- Immaginiamo di costruire un automa che riconosca il linguaggio  $a^n b^m$ , con  $n > 0$  e  $m \geq 0$ 
  - Per creare un automa, ragioniamo su quali simboli possono essere letti
    - Ripetiamo il processo anche sullo stato  $q_1$
    - Cosa succede se leggo 'b' ?
      - Ho bisogno di ricordarlo, creo un nuovo stato (finale)!



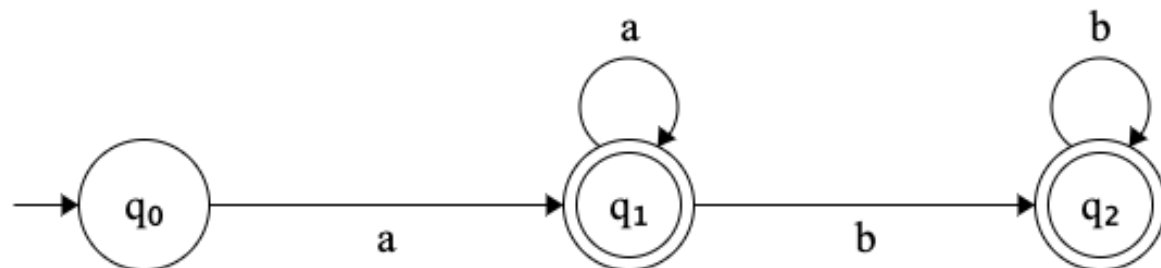
# Un esempio semplicissimo

- Immaginiamo di costruire un automa che riconosca il linguaggio  $a^n b^m$ , con  $n > 0$  e  $m \geq 0$ 
  - Per creare un automa, ragioniamo su quali simboli possono essere letti
    - Ripetiamo il processo anche sullo stato  $q_2$
    - Posso leggere 'a' ?
      - No, perché le parole valide finiscono per b. Le 'b' non possono essere seguite dalle 'a'



# Un esempio semplicissimo

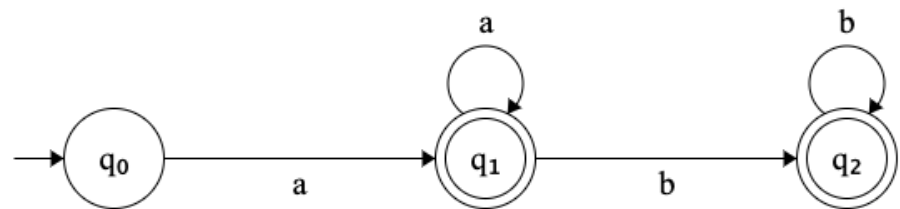
- Immaginiamo di costruire un automa che riconosca il linguaggio  $a^n b^m$ , con  $n > 0$  e  $m \geq 0$ 
  - Per creare un automa, ragioniamo su quali simboli possono essere letti
    - Ripetiamo il processo anche sullo stato  $q_2$
    - Posso leggere 'b' ?
      - Sì, e resto sempre nello stesso stato, seguendo lo stesso ragionamento precedente.



# Un esempio semplicissimo

- Costruiamo la matrice di transizione dell'automa

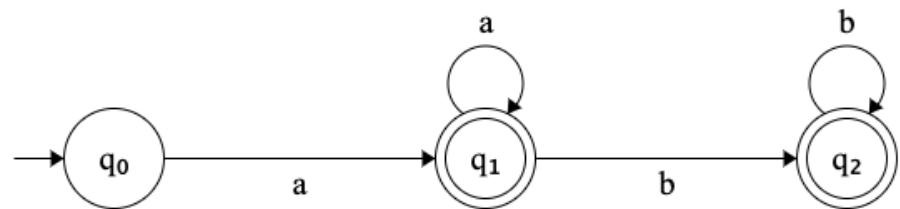
|    | a | b |
|----|---|---|
| q0 |   |   |
| q1 |   |   |
| q2 |   |   |



# Un esempio semplicissimo

- Costruiamo la matrice di transizione dell'automa

|    | a  | b  |
|----|----|----|
| q0 | q1 |    |
| q1 | q1 | q2 |
| q2 |    | q2 |



# Automi per Riconoscere Linguaggi

- Gli automi a stati finiti sono utilizzati per stabilire la correttezza di alcuni frammenti del codice sorgente che scriviamo
- **Esempio, gli identificatori nel linguaggio C**
  - Proviamo a scrivere un automa che riconosca gli identificatori

# Automi per Riconoscere Linguaggi

- Gli automi a stati finiti sono utilizzati per stabilire la correttezza di alcuni frammenti del codice sorgente che scriviamo
- **Esempio, gli identificatori nel linguaggio C**
  - Proviamo a scrivere un automa che riconosca gli identificatori
  - Per prima cosa, descriviamo il Linguaggi degli identificatori. **Quali sono degli identificatori validi?**

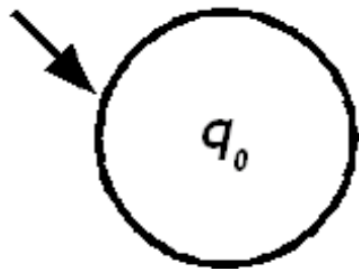
# Automati per Riconoscere Linguaggi

- Gli automi a stati finiti sono utilizzati per stabilire la correttezza di alcuni frammenti del codice sorgente che scriviamo
- **Esempio, gli identificatori nel linguaggio C**
  - Proviamo a scrivere un automa che riconosca gli identificatori
  - Per prima cosa, descriviamo il Linguaggi degli identificatori. **Quali sono degli identificatori validi?**
  - **Un identificatore deve iniziare con una lettera, seguito da un numero arbitrario di lettere e cifre**



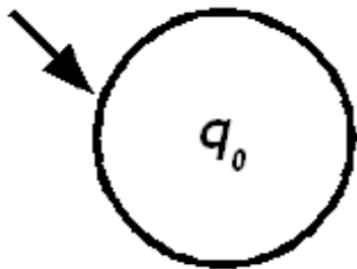
# Automi per Riconoscere Linguaggi

- Gli automi a stati finiti sono utilizzati per stabilire la correttezza di alcuni frammenti del codice sorgente che scriviamo
- **Esempio, gli identificatori nel linguaggio C**



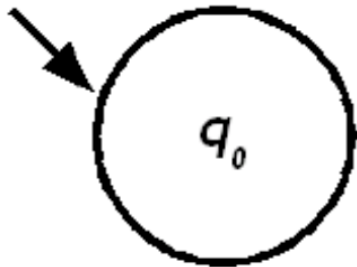
# Automi per Riconoscere Linguaggi

- Gli automi a stati finiti sono utilizzati per stabilire la correttezza di alcuni frammenti del codice sorgente che scriviamo
- **Esempio, gli identificatori nel linguaggio C**
  - Quali sono i simboli che posso leggere?



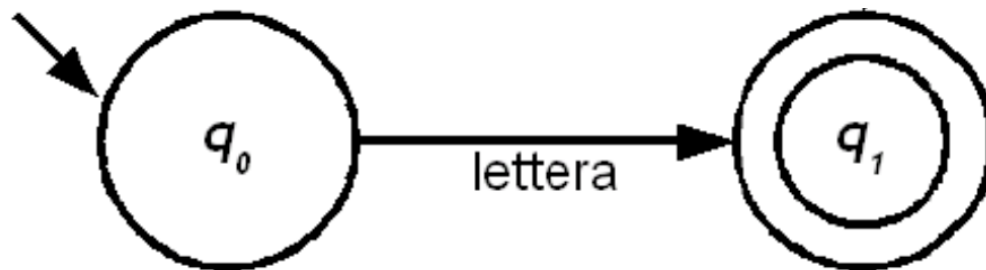
# Automi per Riconoscere Linguaggi

- Gli automi a stati finiti sono utilizzati per stabilire la correttezza di alcuni frammenti del codice sorgente che scriviamo
- **Esempio, gli identificatori nel linguaggio C**
  - Quali sono i simboli che posso leggere? **Lettere e cifre**



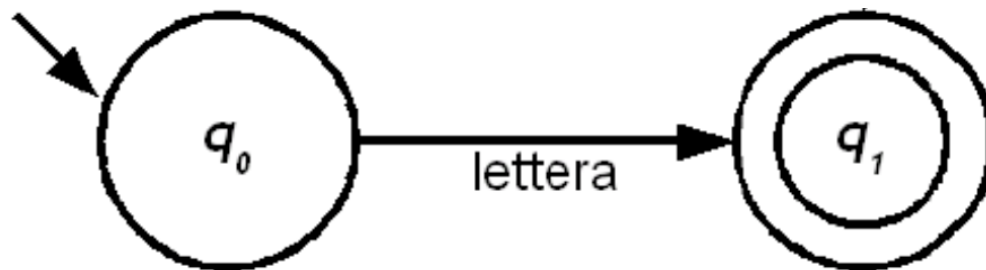
# Automi per Riconoscere Linguaggi

- Gli automi a stati finiti sono utilizzati per stabilire la correttezza di alcuni frammenti del codice sorgente che scriviamo
- **Esempio, gli identificatori nel linguaggio C**
  - Se leggo una lettera, ho un identificatore valido



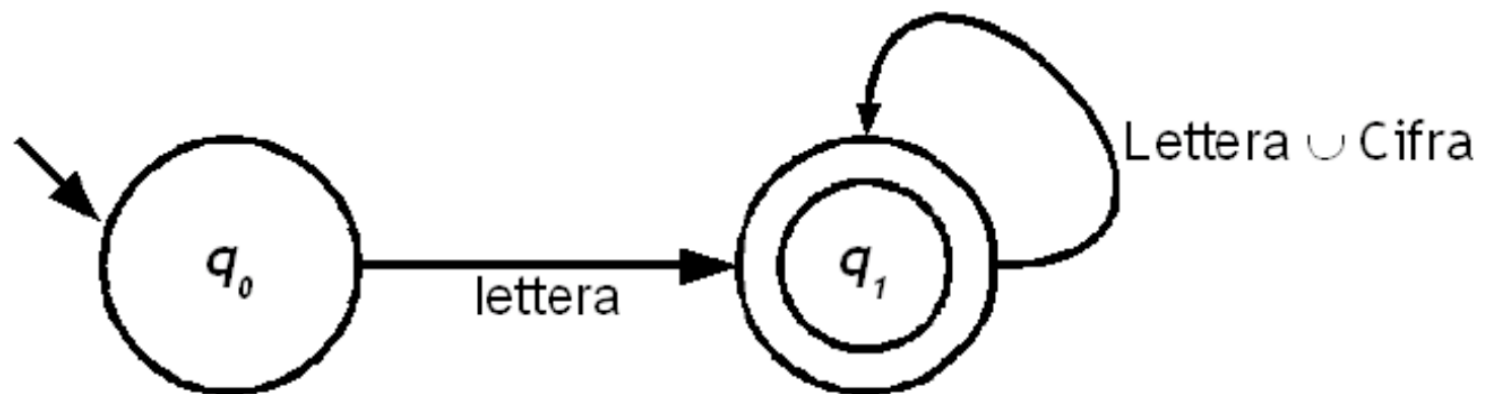
# Automi per Riconoscere Linguaggi

- Gli automi a stati finiti sono utilizzati per stabilire la correttezza di alcuni frammenti del codice sorgente che scriviamo
- **Esempio, gli identificatori nel linguaggio C**
  - Posso farlo seguire da lettere e numeri



# Automi per Riconoscere Linguaggi

- Gli automi a stati finiti sono utilizzati per stabilire la correttezza di alcuni frammenti del codice sorgente che scriviamo
- **Esempio, gli identificatori nel linguaggio C**
  - Posso farlo seguire da lettere e numeri



# Automati per Riconoscere Linguaggi

- Per costruire i vostri automi
  - <https://madebyevan.com/fsm/>
  - **Editor Visuale**

## Estensione della funzione di transizione

- Nella sua definizione di base, la funzione di transizione prende come input uno stato e un carattere dell'alfabeto, e restituisce un nuovo stato.
- **Normalmente però un automa riceve come input una intera stringa, non un carattere.**
- Estendiamo la funzione di transizione a questo scopo.



# Estensione della funzione di transizione

- Si può definire un'estensione della funzione di transizione  $\delta$  come segue:



## Definizione: $\delta^*$ per FSA

Dato un FSA  $M = (Q, \delta, q_0, F)$  con alfabeto di ingresso  $X$ , definiamo per induzione la funzione:

$$\delta^* : Q \times X^* \rightarrow Q$$

- Questa funzione ci dice in **quale stato arriviamo dando in input una stringa** (non un carattere!)

# Estensione della funzione di transizione

- Dato un FSA  $M = (Q, \delta, q_0, F)$  con alfabeto di ingresso  $X$ , definiamo per induzione la funzione:
- tale che  $\delta^*(q, w)$ , per  $\delta^* : Q \times X^* \rightarrow Q$  sia lo stato in cui  $M$  si porta avendo in ingresso la parola  $w$  su  $X$  e partendo dallo stato  $q$ .

La Definizione è Ricorsiva

## Estensione della funzione di transizione

- Dato un FSA  $M = (Q, \delta, q_0, F)$  con alfabeto di ingresso  $X$ , definiamo per induzione la funzione:
- tale che  $\delta^*(q, w)$ , per  $\delta^* : Q \times X^* \rightarrow Q$  sia lo stato in cui  $M$  si porta avendo in ingresso la parola  $w$  su  $X$  e partendo dallo stato  $q$ .

$$\left\{ \begin{array}{l} \delta^*(q, \lambda) = \\ \delta^*(q, xw) = \end{array} \right. \quad \text{per ogni } q \in Q, x \in X, w \in X^*$$

## Estensione della funzione di transizione

- Dato un FSA  $M = (Q, \delta, q_0, F)$  con alfabeto di ingresso  $X$ , definiamo per induzione la funzione:
- tale che  $\delta^*(q, w)$ , per  $\delta^* : Q \times X^* \rightarrow Q$  sia lo stato in cui  $M$  si porta avendo in ingresso la parola  $w$  su  $X$  e partendo dallo stato  $q$ .

$$\left\{ \begin{array}{l} \delta^*(q, \lambda) = q \\ \end{array} \right. \quad \text{per ogni } q \in Q, x \in X, w \in X^*$$

## Estensione della funzione di transizione

- Dato un FSA  $M = (Q, \delta, q_0, F)$  con alfabeto di ingresso  $X$ , definiamo per induzione la funzione:
- tale che  $\delta^*(q, w)$ , per  $\delta^* : Q \times X^* \rightarrow Q$  sia lo stato in cui  $M$  si porta avendo in ingresso la parola  $w$  su  $X$  e partendo dallo stato  $q$ .

$$\begin{cases} \delta^*(q, \lambda) = q \\ \delta^*(q, wx) = \end{cases} \quad \text{per ogni } q \in Q, x \in X, w \in X^*$$

## Estensione della funzione di transizione

- Dato un FSA  $M = (Q, \delta, q_0, F)$  con alfabeto di ingresso  $X$ , definiamo per induzione la funzione:
- tale che  $\delta^*(q, w)$ , per  $\delta^* : Q \times X^* \rightarrow Q$  sia lo stato in cui  $M$  si porta avendo in ingresso la parola  $w$  su  $X$  e partendo dallo stato  $q$ .

$$\begin{cases} \delta^*(q, \lambda) = q \\ \delta^*(q, wx) = \delta(\delta^*(q, w), x) \end{cases} \quad \text{per ogni } q \in Q, x \in X, w \in X^*$$

# Estensione della funzione di transizione

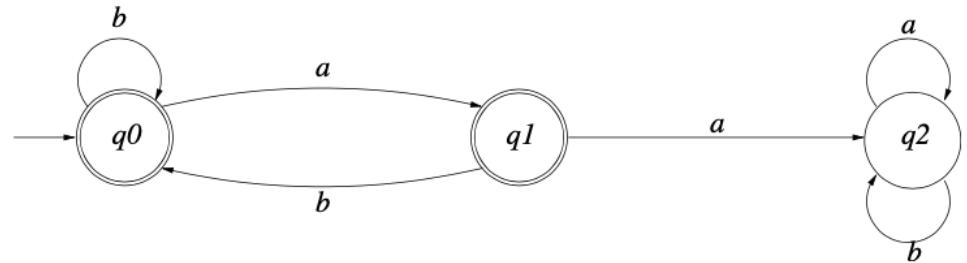
$$\begin{cases} \delta^*(q, \lambda) = q \\ \delta^*(q, wx) = \delta(\delta^*(q, w), x) \end{cases} \quad \text{per ogni } q \in Q, x \in X, w \in X^*$$

- La definizione è ricorsiva
  - Il comportamento della funzione su una stringa è dato **dalla sequenza dei comportamenti della funzione di transizione semplice sui singoli caratteri**
  - Studiamolo con un esempio

# Estensione della funzione di transizione

$$\begin{cases} \delta^*(q, \lambda) = q \\ \delta^*(q, wx) = \delta(\delta^*(q, w), x) \end{cases} \quad \text{per ogni } q \in Q, x \in X, w \in X^*$$

$$\delta^*(q_0, abbb) =$$

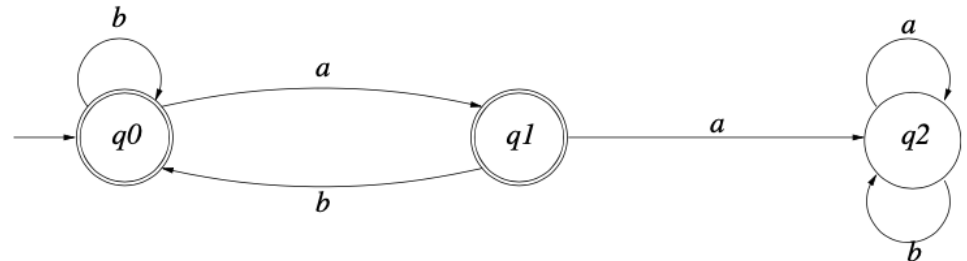




# Estensione della funzione di transizione

$$\begin{cases} \delta^*(q, \lambda) = q \\ \delta^*(q, wx) = \delta(\delta^*(q, w), x) \end{cases} \quad \text{per ogni } q \in Q, x \in X, w \in X^*$$

$$\delta^*(q_0, \underbrace{abbb}_w x) =$$



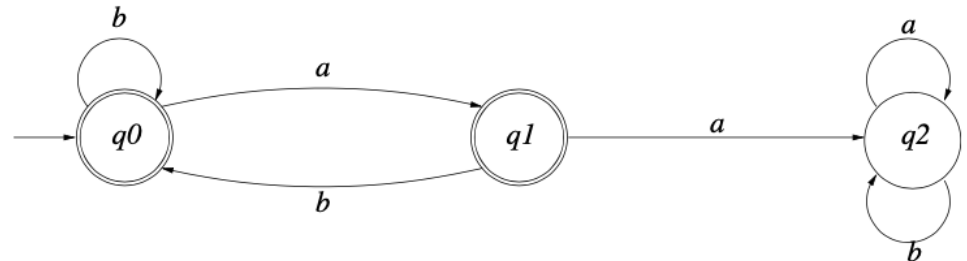
Applichiamo la definizione

# Estensione della funzione di transizione

$$\begin{cases} \delta^*(q, \lambda) = q \\ \delta^*(q, wx) = \delta(\delta^*(q, w), x) \end{cases} \quad \text{per ogni } q \in Q, x \in X, w \in X^*$$

$$\delta^*(q_0, abbb) =$$

$$\delta(\delta^*(q_0, abb)b) =$$



# Estensione della funzione di transizione

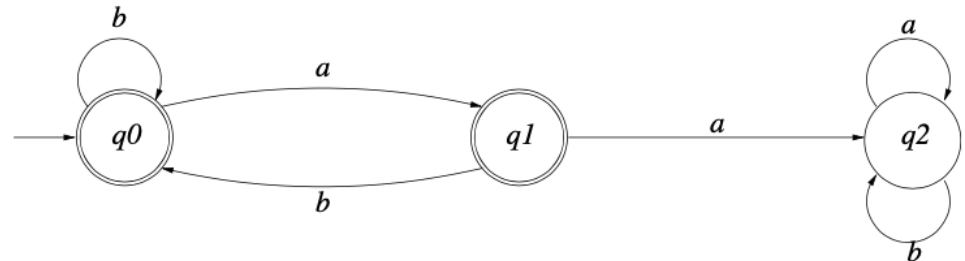
$$\begin{cases} \delta^*(q, \lambda) = q \\ \delta^*(q, wx) = \delta(\delta^*(q, w), x) \end{cases}$$

per ogni  $q \in Q$ ,  $x \in X$ ,  $w \in X^*$

$$\delta^*(q_0, abbb) =$$

$$\delta(\delta^*(q_0, \underbrace{abb}_w)b) =$$

$w$        $x$



# Estensione della funzione di transizione

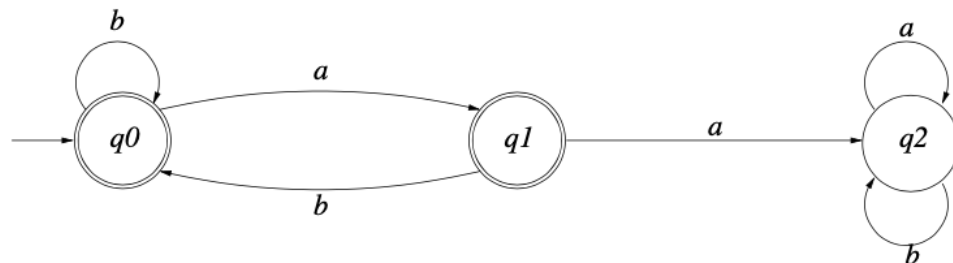
$$\begin{cases} \delta^*(q, \lambda) = q \\ \delta^*(q, wx) = \delta(\delta^*(q, w), x) \end{cases} \quad \text{per ogni } q \in Q, x \in X, w \in X^*$$

$$\delta^*(q_0, abbb) =$$

$$\delta(\delta^*(q_0, abb)b) =$$

$$\delta(\delta(\delta^*(q_0, ab)b)b) =$$

$$\delta(\delta(\delta(\delta(q_0, a)b)b)b) =$$



# Estensione della funzione di transizione

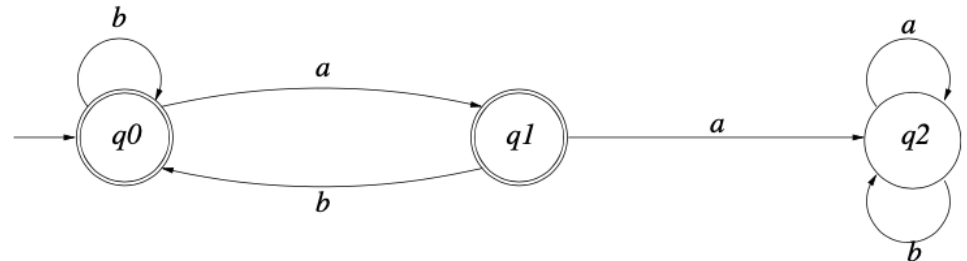
$$\begin{cases} \delta^*(q, \lambda) = q \\ \delta^*(q, wx) = \delta(\delta^*(q, w), x) \end{cases} \quad \text{per ogni } q \in Q, x \in X, w \in X^*$$

$$\delta^*(q_0, abbb) =$$

$$\delta(\delta^*(q_0, abb)b) =$$

$$\delta(\delta(\delta^*(q_0, ab)b)b) =$$

$$\delta(\delta(\delta(\delta(q_0, a)b)b)b) =$$



La funzione di transizione estesa non è altro che **una sequenza di chiamate ricorsive alla funzione di transizione semplice**

# Estensione della funzione di transizione

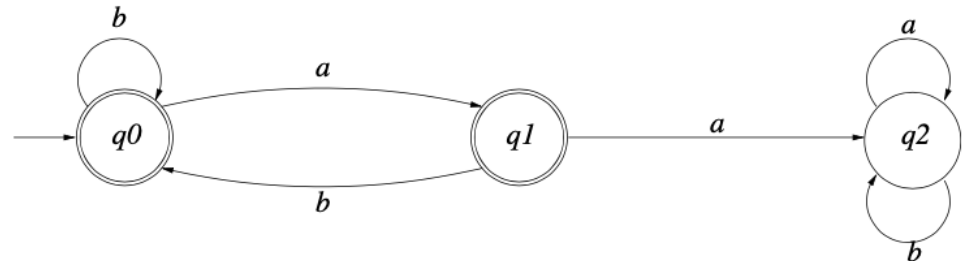
$$\begin{cases} \delta^*(q, \lambda) = q \\ \delta^*(q, wx) = \delta(\delta^*(q, w), x) \end{cases} \quad \text{per ogni } q \in Q, x \in X, w \in X^*$$

$$\delta^*(q_0, abbb) =$$

$$\delta(\delta^*(q_0, abb)b) =$$

$$\delta(\delta(\delta^*(q_0, ab)b)b) =$$

$$\delta(\delta(\delta(\delta(q_0, a)b)b)b)$$

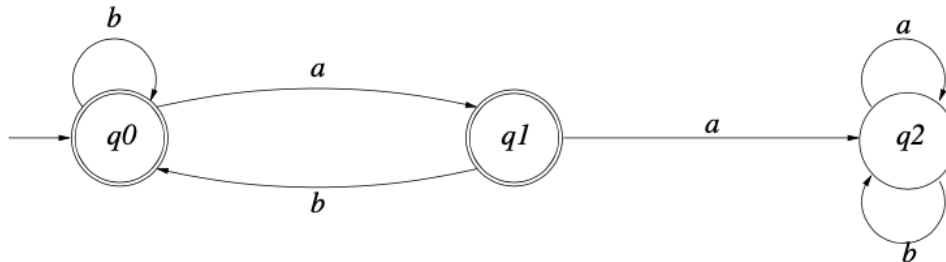


Sviluppiamola al contrario  
per trovare la soluzione

# Estensione della funzione di transizione

$$\begin{cases} \delta^*(q, \lambda) = q \\ \delta^*(q, wx) = \delta(\delta^*(q, w), x) \end{cases} \quad \text{per ogni } q \in Q, x \in X, w \in X^*$$

$$\delta^*(q_0, abbb) =$$
$$\delta(q_0, a) =$$



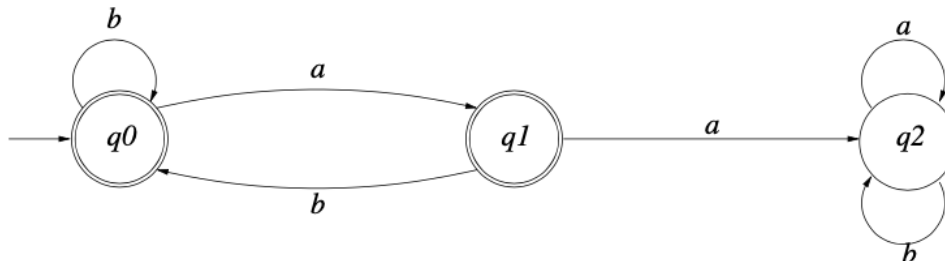
# Estensione della funzione di transizione

$$\begin{cases} \delta^*(q, \lambda) = q \\ \delta^*(q, wx) = \delta(\delta^*(q, w), x) \end{cases} \quad \text{per ogni } q \in Q, x \in X, w \in X^*$$

$$\delta^*(q_0, abbb) =$$

$$\delta(q_0, a) = q_1 \rightarrow$$

Lo stato di output della prima chiamata **diventa lo stato di input della seconda** (con il secondo carattere, ovviamente)

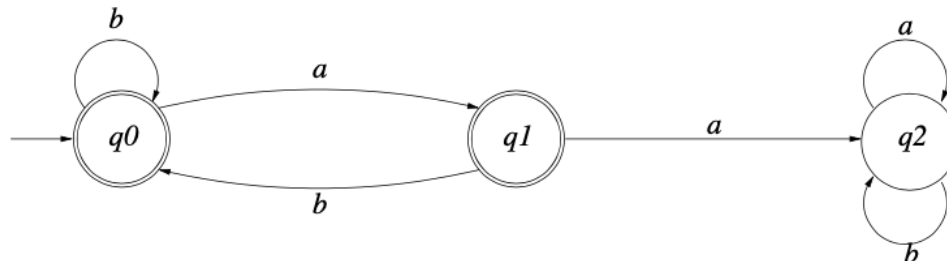




# Estensione della funzione di transizione

$$\begin{cases} \delta^*(q, \lambda) = q \\ \delta^*(q, wx) = \delta(\delta^*(q, w), x) \end{cases} \quad \text{per ogni } q \in Q, x \in X, w \in X^*$$

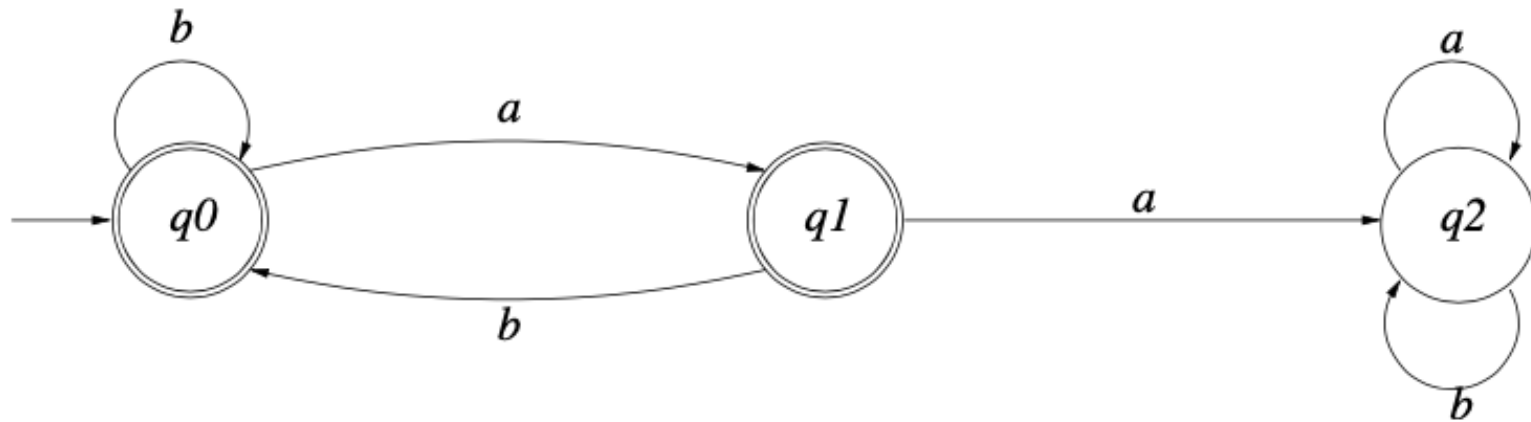
$$\begin{aligned} \delta^*(q_0, abbb) = \\ \delta(q_0, a) = q_1 \rightarrow \\ \delta(q_1, b) = q_0 \\ \delta(q_0, b) = q_0 \\ \delta(q_0, b) = q_0 \end{aligned}$$



## Estensione della funzione di transizione

$$\begin{cases} \delta^*(q, \lambda) = q \\ \delta^*(q, wx) = \delta(\delta^*(q, w), x) \end{cases} \quad \text{per ogni } q \in Q, x \in X, w \in X^*$$

$$\delta^*(q_0, abbb) = q_0$$



# Parola accettata o riconosciuta da un FSA

## ■ Definizione

Sia  $M = (Q, \delta, q_0, F)$  un FSA con alfabeto di ingresso  $X$ . Una *parola*  $w \in X^*$  è **accettata** (o **riconosciuta**) da  $M$  se, partendo dallo stato iniziale  $q_0$ , lo stato  $q$  in cui l'automa si porta alla fine della sequenza di ingresso  $w$  è uno stato finale.

$$w \text{ accettata} \stackrel{\text{def}}{\iff} \delta^*(q_0, w) \in F$$

$$\delta^* : Q \times X^* \rightarrow Q$$

L'output della funzione  
è uno stato!

# Linguaggio accettato o riconosciuto da un FSA

## ■ Definizione

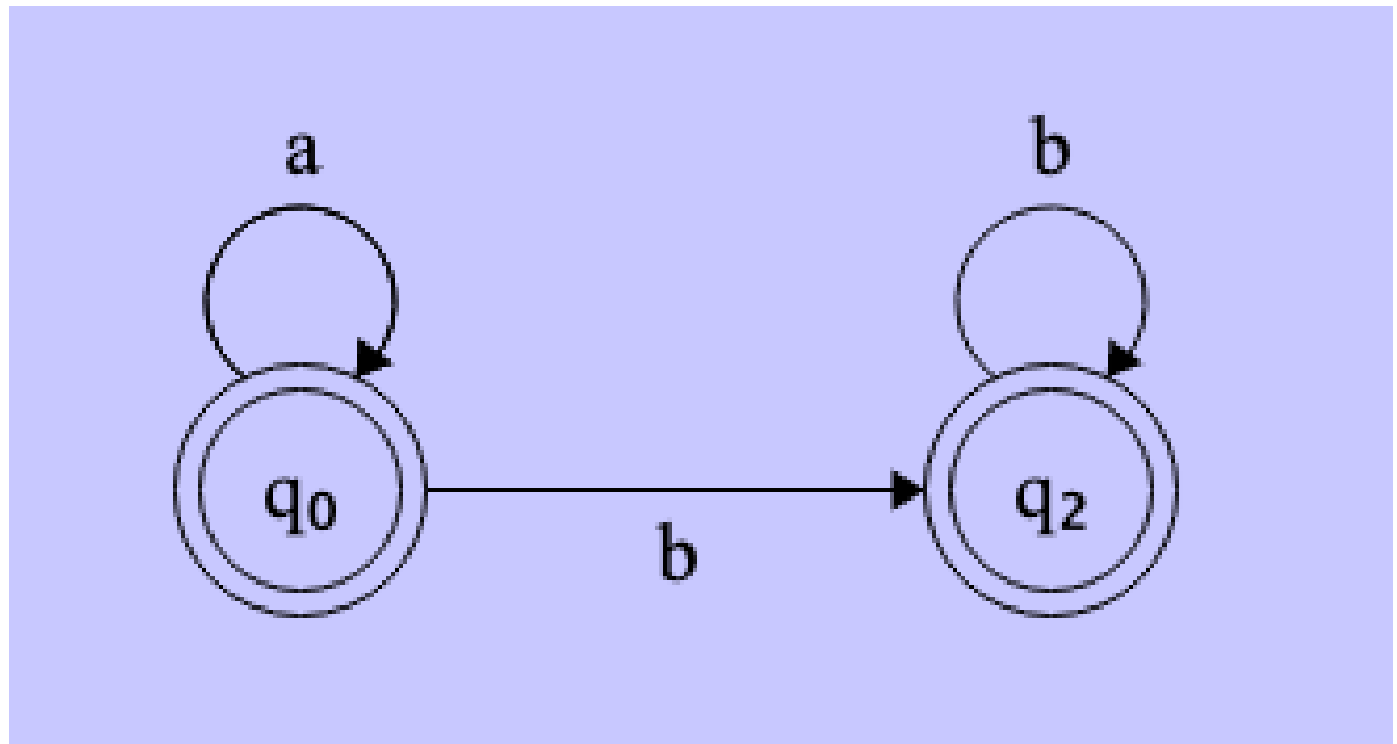
Sia  $M = (Q, \delta, q_0, F)$  un FSA con alfabeto di ingresso  $X$ . Il *linguaggio accettato* o *riconosciuto* da  $M$  è il seguente sottoinsieme di  $X^*$ :

$$T(M) = \{w \in X^* \mid \delta^*(q_0, w) \in F\}$$

(l'insieme delle parole accettate da  $M$ ).

# Linguaggio accettato o riconosciuto da un FSA

## ■ Esempio



$$T(M) = a^n b^m, n \geq 0, m > 0$$

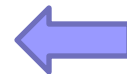
# Linguaggio accettato o riconosciuto da un FSA

## ■ Esempio

### ■ Esempio

$$T(M) = a^n b^m, n \geq 0, m \geq 0$$

$$T(M) = a^n b^m, n \geq 0, m \geq 0$$



Si utilizza un descrittore!

# FSA equivalenti

## ■ Definizione

Sia  $M_1 = (Q_1, \delta_1, q_1, F_1)$  ed  $M_2 = (Q_2, \delta_2, q_2, F_2)$  due FSA di alfabeto di ingresso  $X$ .  $M_1$  ed  $M_2$  si dicono *equivalenti* se:

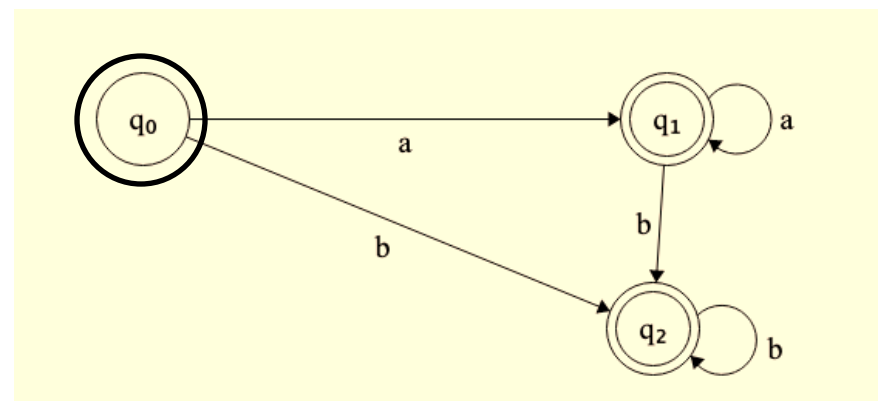
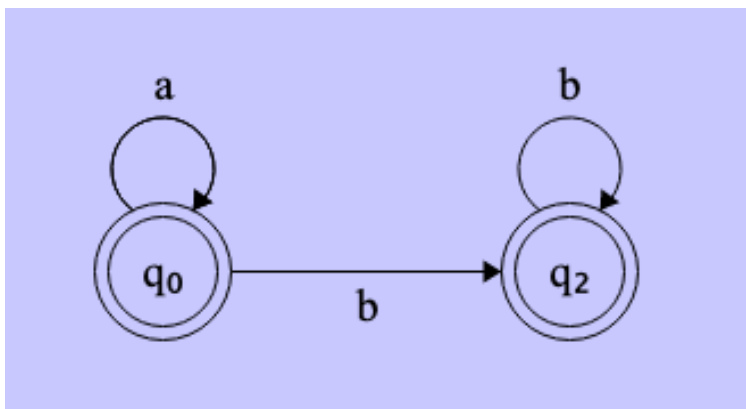
$$T(M_1) = T(M_2)$$

# FSA equivalenti

## ■ Definizione

Sia  $M_1 = (Q_1, \delta_1, q_1, F_1)$  ed  $M_2 = (Q_2, \delta_2, q_2, F_2)$  due FSA di alfabeto di ingresso  $X$ .  $M_1$  ed  $M_2$  si dicono *equivalenti* se:

$$T(M_1) = T(M_2)$$





# Linguaggi a stati finiti

## ■ Definizione

Dato un alfabeto  $X$ , un linguaggio  $L$  su  $X$  è un *linguaggio a stati finiti* (o FSL - Finite State Language) se esiste un FSA  $M$  con alfabeto di ingresso  $X$  tale che  $L = T(M)$ .

# Classe dei linguaggi a stati finiti

## ■ Definizione

Di seguito la definizione della *Classe dei Linguaggi a Stati Finiti*

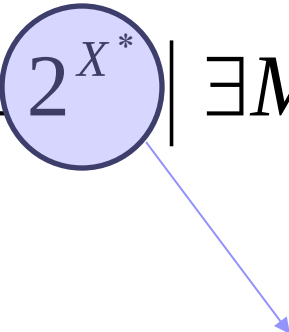
$$\mathcal{L}_{FSL} = \left\{ L \in 2^{X^*} \mid \exists M, M \text{ è un FSA} : L = T(M) \right\}$$

# Classe dei linguaggi a stati finiti

## ■ Definizione

Di seguito la definizione della *Classe dei Linguaggi a Stati Finiti*

$$\mathcal{L}_{FSL} = \left\{ L \in 2^{X^*} \mid \exists M, M \text{ è un FSA} : L = T(M) \right\}$$



Insieme di tutti i **possibili linguaggi**

Si tratta dell'insieme dei linguaggi **per i quali è possibile costruire un automa che li riconosca**

# Esercizio

**Esercizio 1.** Costruire un FSA che accetti questo linguaggio:  
 $L = \{w \in \{a, b\}^* \mid w \text{ ha un numero pari di } a \text{ e dispari di } b\}$

# Esercizio

**Esercizio 1.** Costruire un FSA che accetti questo linguaggio:

$$L = \{w \in \{a, b\}^* \mid w \text{ ha un numero pari di } a \text{ e dispari di } b\}$$

**Soluzione:** Sia  $M = (Q, \delta, q_0, F) \in \text{FSA}$

- $Q =$

# Esercizio

**Esercizio 1.** Costruire un FSA che accetti questo linguaggio:

$$L = \{w \in \{a, b\}^* \mid w \text{ ha un numero pari di } a \text{ e dispari di } b\}$$

**Soluzione:** Sia  $M = (Q, \delta, q_0, F) \in \text{FSA}$

- $Q =$

**Suggerimento:** gli stati codificano possibili «situazioni» che abbiamo bisogno di ricordare. In questo caso cosa ci serve «ricordare» per capire se una parola è corretta oppure no?

# Esercizio

**Esercizio 1.** Costruire un FSA che accetti questo linguaggio:

$$L = \{w \in \{a, b\}^* \mid w \text{ ha un numero pari di } a \text{ e dispari di } b\}$$

**Soluzione:** Sia  $M = (Q, \delta, q_0, F) \in \text{FSA}$

- $Q =$

**Suggerimento:** gli stati codificano possibili «situazioni» che abbiamo bisogno di ricordare. In questo caso cosa ci serve «ricordare» per capire se una parola è corretta oppure no?

Ci serve ricordare il numero di  $a$  e il numero di  $b$  (le « $a$ » devono essere pari e le « $b$ » dispari).

Abbiamo però un problema, perché sappiamo che gli stati sono finiti, mentre invece le  $a$  e le  $b$  possono essere infinite.

**Come procediamo?**

# Esercizio

**Esercizio 1.** Costruire un FSA che accetti questo linguaggio:

$$L = \{w \in \{a, b\}^* \mid w \text{ ha un numero pari di } a \text{ e dispari di } b\}$$

**Soluzione:** Sia  $M = (Q, \delta, q_0, F) \in \text{FSA}$

- $Q =$

**Suggerimento:** gli stati codificano possibili «situazioni» che abbiamo bisogno di ricordare. In questo caso cosa ci serve «ricordare» per capire se una parola è corretta oppure no?

Ci serve ricordare il numero di  $a$  e il numero di  $b$  (le « $a$ » devono essere pari e le « $b$ » dispari).

**Non ci serve sapere il numero esatto di  $a$  e  $b$ , ci basta sapere se sono pari o sono dispari!**



# Esercizio

**Esercizio 1.** Costruire un FSA che accetti questo linguaggio:

$$L = \{w \in \{a, b\}^* \mid w \text{ ha un numero pari di } a \text{ e dispari di } b\}$$

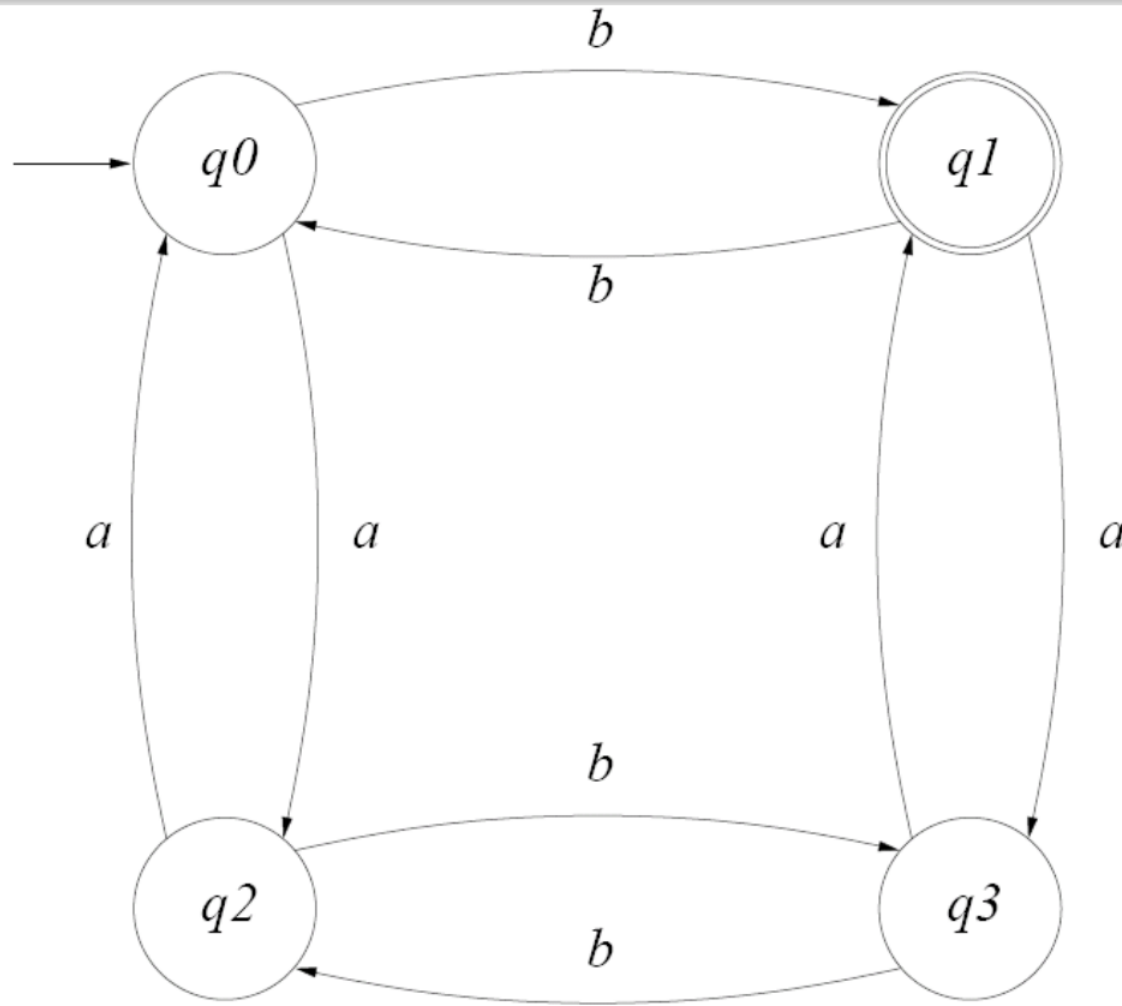
**Soluzione:** Sia  $M = (Q, \delta, q_0, F) \in \text{FSA}$

- $Q = \{q_0, q_1, q_2, q_3\}$  dove
  - $q_0$  stato per un numero pari di  $a$  e di  $b$
  - $q_1$  stato per un numero pari di  $a$  e dispari di  $b$
  - $q_2$  stato per un numero dispari di  $a$  e pari di  $b$
  - $q_3$  stato per un numero dispari di  $a$  e di  $b$

# Esercizio

- Costruiamo l'automa

# Esercizio – Costruiamo l'Automa



## Un altro esempio

**Esercizio 2.** Costruire un FSA che accetti questo linguaggio:

$$L = \{w \in \{a, b\}^* \mid w \neq \alpha aa\beta, \alpha, \beta \in \{a, b\}^*\}$$

## Un altro esempio

**Esercizio 2.** Costruire un FSA che accetti questo linguaggio:

$$L = \{w \in \{a, b\}^* \mid w \neq \alpha aa\beta, \alpha, \beta \in \{a, b\}^*\}$$

$\alpha, \beta = \text{qualsiasi stringa}$

## Un altro esempio

**Esercizio 2.** Costruire un FSA che accetti questo linguaggio:

$$L = \{w \in \{a, b\}^* \mid w \neq \alpha aa\beta, \alpha, \beta \in \{a, b\}^*\}$$

$\alpha, \beta = \text{qualsiasi stringa}$

**W non deve contenere la sottostringa 'aa'**

## Un altro esempio

**Esercizio 2.** Costruire un FSA che accetti questo linguaggio:

$$L = \{w \in \{a, b\}^* \mid w \neq \alpha aa\beta, \alpha, \beta \in \{a, b\}^*\}$$

$\alpha, \beta = \text{qualsiasi stringa}$

**W non deve contenere la sottostringa 'aa'**

Significa che posso leggere qualsiasi cosa, ma se leggo «aa» la parola non fa più parte del Linguaggio

## Un altro esempio

**Esercizio 2.** Costruire un FSA che accetti questo linguaggio:

$$L = \{w \in \{a, b\}^* \mid w \neq \alpha aa\beta, \alpha, \beta \in \{a, b\}^*\}$$

**Soluzione:** Sia  $M = (Q, \delta, q_0, F) \in \text{FSA}$

- $Q = \{q_0, q_1, q_2\}$  dove



## Un altro esempio

**Esercizio 2.** Costruire un FSA che accetti questo linguaggio:

$$L = \{w \in \{a, b\}^* \mid w \neq \alpha aa\beta, \alpha, \beta \in \{a, b\}^*\}$$

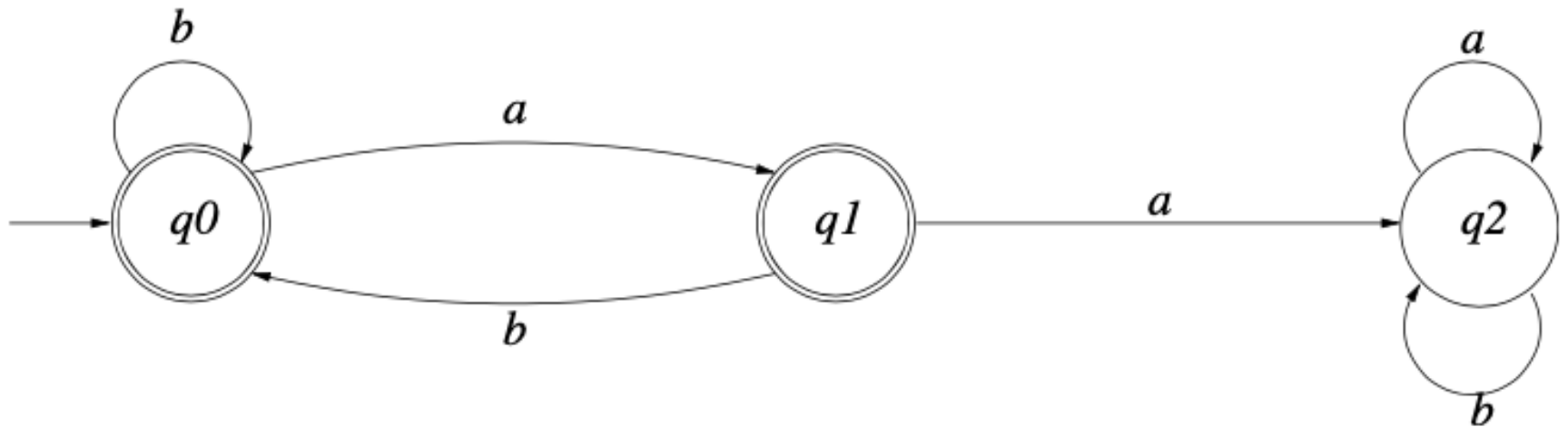
**Soluzione:** Sia  $M = (Q, \delta, q_0, F) \in \text{FSA}$

- $Q = \{q_0, q_1, q_2\}$  dove
  - **q0** = stato in cui non ho mai letto una 'a'
  - **q1** = stato in cui ho letto la prima 'a'
  - **q2** = stato in cui ho letto le due 'a' consecutive  
(*non di accettazione*)

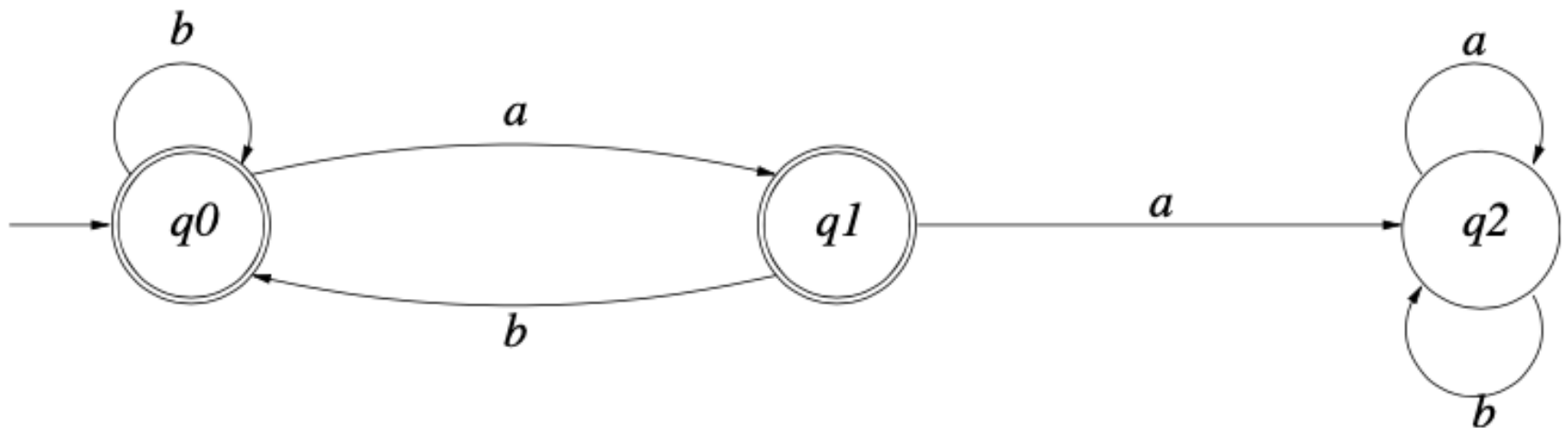


# Costruiamo l'automa

## Costruiamo l'automa



## Costruiamo l'automa



- funzione di transizione  $\delta$  definita:
  - $\delta(q_0, a) = q_1$
  - $\delta(q_0, b) = \delta(q_1, b) = q_0$
  - $\delta(q_1, a) = \delta(q_2, a) = \delta(q_2, b) = q_2$
- $q_0$  stato iniziale
- $F = \{q_0, q_1\}$

# Proposizione

- I linguaggi a stati finiti sono *chiusi* rispetto al *complemento*.

## Dimostrazione

Sia  $L \in \mathcal{L}_{FSL}$  un linguaggio a stati finiti sull'alfabeto  $X$ .  
Dalla definizione di linguaggio a stati finiti,  $L = T(M)$ ,  
ove  $M = (Q, \delta, q_0, F)$ .

Consideriamo il complemento di  $L$ :  $\bar{L} = X^* - L$  e  
l'automa a stati finiti  $\bar{M} = (Q, \delta, q_0, Q - F)$ . Si ha:  $\bar{L} = T(\bar{M})$   
(si dimostra per induzione sulla lunghezza di una parola  $w$ ).

## Esercizio - Complemento

**1** Data  $R = a(aa)^*b^*$  definire un DFA  $M'$  tale che  $L(M') = \overline{L(R)}$

### Risoluzione consigliata

1. Costruire prima l'automa che riconosce  $a(aa)^*b^*$
2. Rendere la funzione di transizione **completa**
3. Costruire l'automa complemento invertendo gli stati

# Esercizio - Complemento

**1** Data  $R = a(aa)^*b^*$  definire un DFA  $M'$  tale che  $L(M') = \overline{L(R)}$

## Risoluzione consigliata

1. Costruire prima l'automa che riconosce  $a(aa)^*b^*$

# Esercizio - Complemento

**1** Data  $R = a(aa)^*b^*$  definire un DFA  $M'$  tale che  $L(M') = \overline{L(R)}$

## Risoluzione consigliata

1. Costruire prima l'automa che riconosce  $a(aa)^*b^*$

- Numero dispari di  $a$
- Seguito eventualmente da una sequenza di  $b$
- Elencate le parole del linguaggio, sempre!
- $T(M) = \{a, aaa, aaaaa, ab, abbb, abb, aaabb, aaabbbbbb, etc.\}$

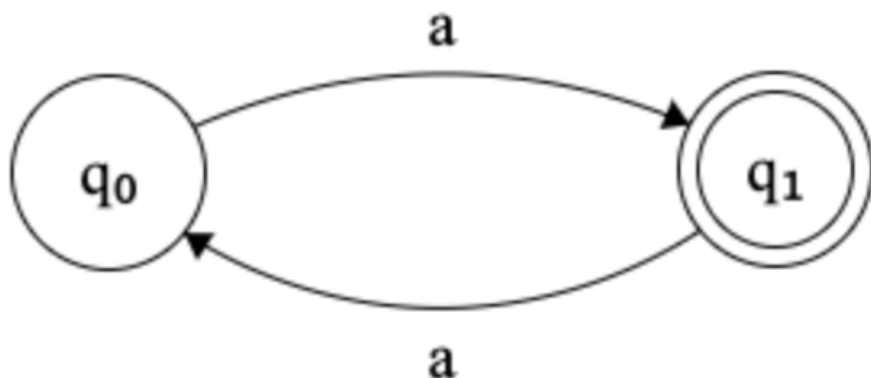


## Esercizio - Complemento

**1** Data  $R = a(aa)^*b^*$  definire un DFA  $M'$  tale che  $L(M') = \overline{L(R)}$

### Risoluzione consigliata

1. Costruire prima l'automa che riconosce  $a(aa)^*b^*$ 
  - Numero dispari di  $a$



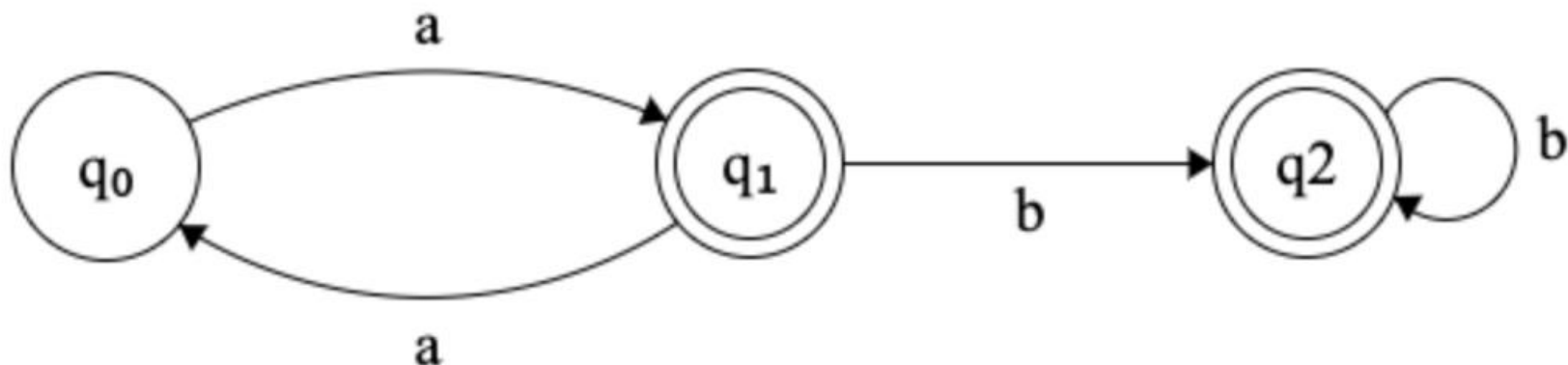
## Esercizio - Complemento

**1** Data  $R = a(aa)^*b^*$  definire un DFA  $M'$  tale che  $L(M') = \overline{L(R)}$

### Risoluzione consigliata

1. Costruire prima l'automa che riconosce  $a(aa)^*b^*$

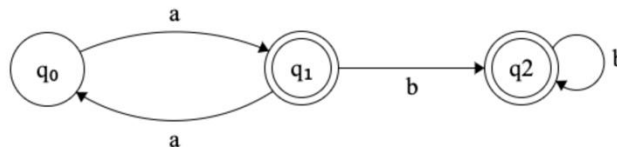
- Numero dispari di  $a$
- Seguito eventualmente da una sequenza di  $b$



# Esercizio - Complemento

**1** Data  $R = a(aa)^*b^*$  definire un DFA  $M'$  tale che  $L(M') = \overline{L(R)}$

Rendiamo la funzione di transizione totale, aggiungendo uno stato pozza.

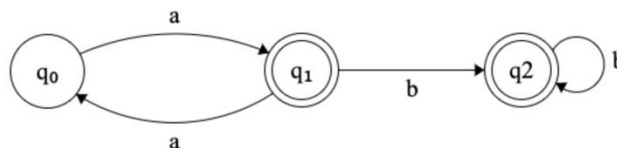


|       | a | b |
|-------|---|---|
| $q_0$ |   |   |
| $q_1$ |   |   |
| $q_2$ |   |   |

# Esercizio - Complemento

**1** Data  $R = a(aa)^*b^*$  definire un DFA  $M'$  tale che  $L(M') = \overline{L(R)}$

Rendiamo la funzione di transizione totale, aggiungendo uno stato pozza.

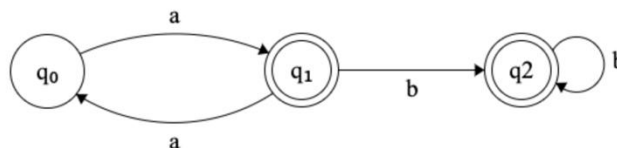


|    | a  | b  |
|----|----|----|
| q0 | q1 |    |
| q1 | q0 | q2 |
| q2 |    | q2 |

# Esercizio - Complemento

**1** Data  $R = a(aa)^*b^*$  definire un DFA  $M'$  tale che  $L(M') = \overline{L(R)}$

Rendiamo la funzione di transizione totale, aggiungendo uno stato pozza.

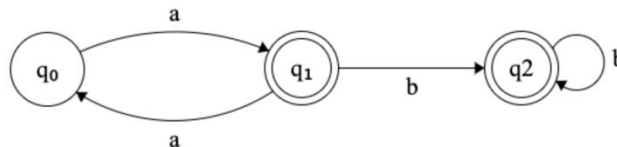


|    | a  | b  |
|----|----|----|
| q0 | q1 | X  |
| q1 | q0 | q2 |
| q2 | X  | q2 |

# Esercizio - Complemento

**1** Data  $R = a(aa)^*b^*$  definire un DFA  $M'$  tale che  $L(M') = \overline{L(R)}$

Rendiamo la funzione di transizione totale, aggiungendo uno stato pozza.

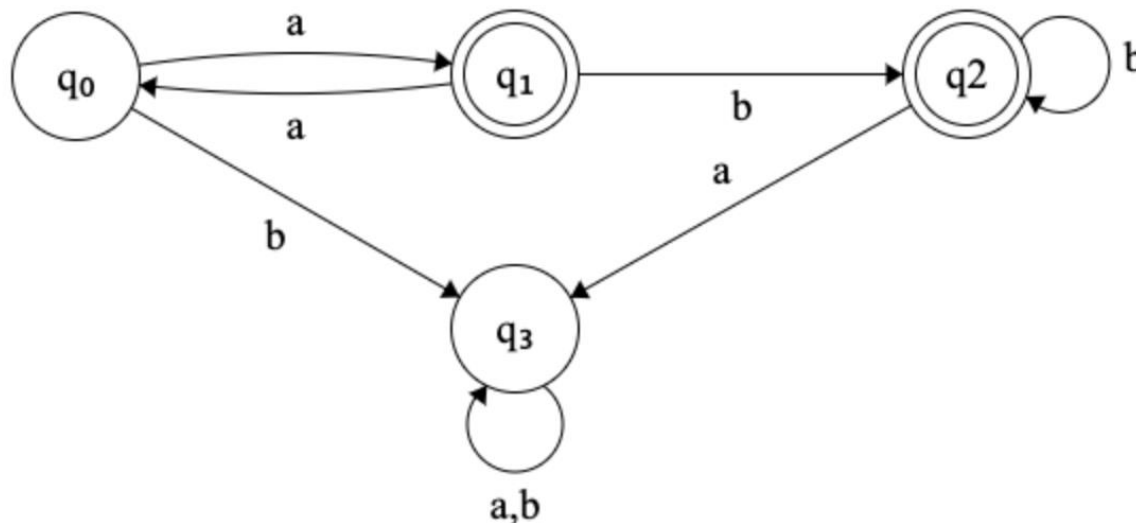


|    | a         | b         |
|----|-----------|-----------|
| q0 | q1        | <b>q3</b> |
| q1 | q0        | q2        |
| q2 | <b>q3</b> | q2        |
| q3 | <b>q3</b> | <b>q3</b> |

# Esercizio - Complemento

**1** Data  $R = a(aa)^*b^*$  definire un DFA  $M'$  tale che  $L(M') = \overline{L(R)}$

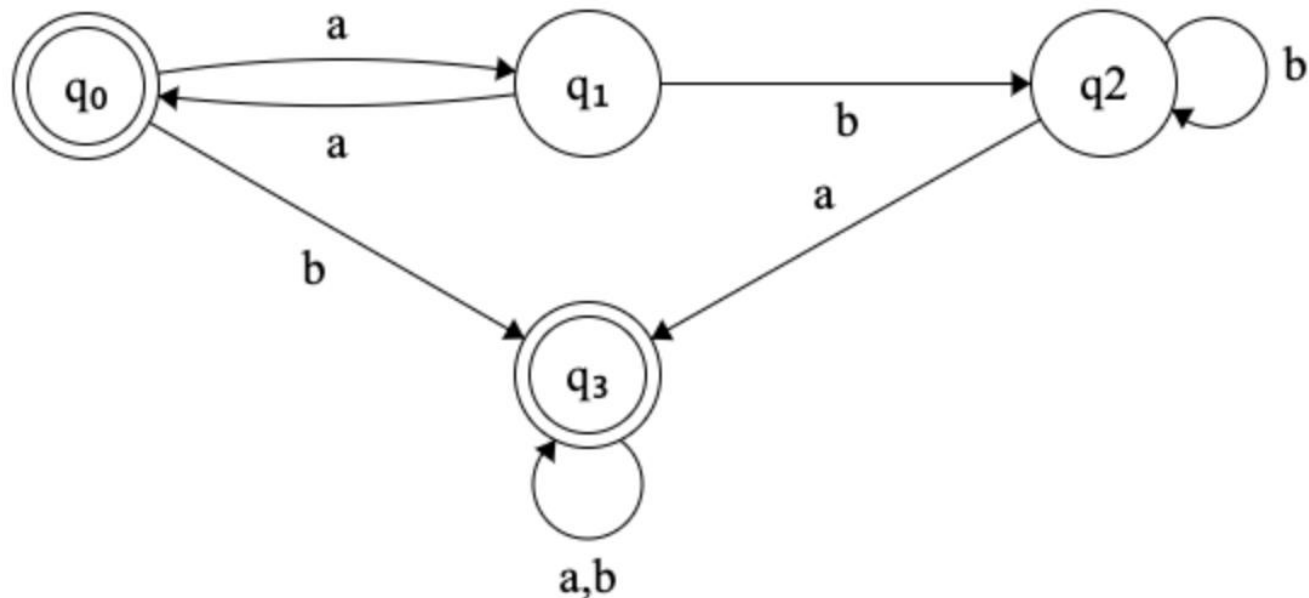
Rendiamo la funzione di transizione totale, aggiungendo uno stato pozza.



# Esercizio - Complemento

**1** Data  $R = a(aa)^*b^*$  definire un DFA  $M'$  tale che  $L(M') = \overline{L(R)}$

Inversione degli stati per riconoscere il complemento





## Domande?



*Bojack, when you get sad, you run. Straight ahead, and you keep running forward no matter what. There are people in your life who are gonna try to hold you back, slow you down, but you don't let them. Don't you stop running and don't you ever look behind you, there's nothing for you behind you. All that exists is what's ahead.*

*-Secretariat*